A New Approach to Reducing Modulus Requirements in Homomorphic Computation: Precision Allocation in CKKS

Zhaoyang Liang[1,3]
Nankai University

Runkun Liu[2,3]
Nankai University

Jingwei Fang[1]
Nankai University

Dan Ding[1,3,4]
Nankai University
Serica Laboratory

Abstract

Fully homomorphic encryption (FHE) enables computation over encrypted data subject to the prescribed security and precision requirements (allowable deviations from ideal results). Among all FHE schemes, CKKS is a representative scheme for approximate arithmetic, allowing the same operation on multiple values packed into a single RLWE ciphertext.

Reducing the ciphertext modulus Q for given output precision requirements under RLWE security constraints is a central challenge in homomorphic computation. However, despite advances in plaintext and ciphertext representations, algorithmic optimizations, and modulus management, most existing approaches still base their modulus budgets on the highest required precision and cannot reduce the base modulus cost of overcoming encryption noise.

In this work, we make *average* rather than *maximum* precision requirements govern the modulus budget and achieve savings in the baseline cost. Under decryption correctness constraints, we make this possible by allocating *arbitrary* output precision among independent values and quantify the modulus savings arising from the gap between precision requirements.

As a bonus, our construction preserves the standard CKKS ciphertext format, which helps maintain compatibility with existing optimizations. And the construction inherits the IND-CPA security of CKKS without additional cryptographic assumptions. In a repeated squaring experiment, we use 67% fewer initial modulus bits and one quarter of the ring dimension of standard CKKS, while meeting the same precision and security requirements.

[1]College of Cryptology and Cyber Sciences, Nankai University, Tianjin 300350, P. R. China. Emails: liangzhaoyang@mail.nankai.edu.cn, jingweifang@mail.nankai.edu.cn, dingdan@nankai.edu.cn.

[2]School of Mathematical Sciences, Nankai University, Tianjin 300071, P. R. China. Email: liurunkun@mail.nankai.edu.cn.

[3]College of Computer Sciences, Nankai University, Tianjin 300350, P. R. China. Emails: liangzhaoyang@mail.nankai.edu.cn, liurunkun@mail.nankai.edu.cn, dingdan@nankai.edu.cn.

[4]Serica Laboratory, Beijing 100084, P. R. China. Email: dingdan@serica-tech.com.cn.

Contents

1 Introduction **1**
- 1.1 Our Results and Techniques . . . 2
- 1.2 Related Work . . . 3

2 Preliminaries **4**
- 2.1 Notations . . . 4
- 2.2 CKKS Ciphertexts and Precision . . . 4
- 2.3 Units and Logarithmic Embeddings . . . 4

3 Precision Allocation via Ring Multipliers **5**
- 3.1 Encoding and Message Recovery . . . 5

4 Construction of Ring Multipliers **7**
- 4.1 From Precision Requirements to Embedding Weights . . . 7
- 4.2 Explicit Construction via Fourier Inversion . . . 7

5 From Precision Allocation to Resource Savings **9**
- 5.1 Scale and Modulus Reduction . . . 9

6 Precision Allocation under Recovery Constraints **10**
- 6.1 Effects of Precision Differences on Error . . . 10
- 6.2 Allocation with Dummy Slots . . . 12

7 Scheme Analysis **13**
- 7.1 Homomorphic Evaluation . . . 13
- 7.2 Error Analysis and Security . . . 15
- 7.3 Computation and Storage Costs . . . 16

8 Implementation **18**
- 8.1 Precision Transfer and Recoverable Depth . . . 18
- 8.2 Parameter and Time Savings . . . 19
- 8.3 Transfer Amount and Message Count . . . 20
- 8.4 Zero and Negative Average Precision Targets . . . 21
- 8.5 Comparison with Tuple CKKS . . . 22

A Deferred Proofs **23**

B Additional Error Propagation **25**

C Details of RNS Key Switching **26**

D Bootstrapping with Precision Allocation and Ring Switching **27**
- D.1 Adapting the Bootstrapping Steps . . . 27
- D.2 Experiments . . . 30

E A Concrete Example of the Error Bounds **32**

F Concurrent Work **36**

1 Introduction

Fully homomorphic encryption (FHE) enables arbitrary computation on encrypted data without revealing the underlying messages. In Gentry's first construction [Gen09, STOC 2009] and subsequent lattice-based constructions [BV11, BGV12, Bra12], efficiency depends in part on controlling the noise introduced by encryption and homomorphic operations. The ciphertext modulus Q limits how much homomorphic computation can be performed at a given precision. Increasing Q generally supports higher precision, but it also increases ciphertext storage and computational costs. With other parameters fixed, an excessively large Q can also compromise security. Reducing the modulus required for a computation is therefore a central challenge in improving the efficiency of homomorphic computation.

We study this question for CKKS [CKKS17], which supports approximate arithmetic on real and complex numbers. CKKS packs a vector $\mathbf{x} = (x_0, \ldots, x_{r-1})$ into a single ring-learning-with-errors (RLWE) ciphertext [LPR10, PRSD17]. The coordinates $x_j, j \in \{0, ..., r-1\}$, called slots, support coordinatewise addition and multiplication [SV14]. All slots share a common scale Δ, used to reduce the effect of noise and meet the precision requirements. Absolute precision of b bits means $|x_j - \widehat{x}_j| \leq 2^{-b}$ in each slot, where $\widehat{x}_j$ is the value recovered after decryption and decoding. If E bounds the error in each slot before division by Δ, this precision is guaranteed whenever

$$\frac{E}{\Delta} \leq 2^{-b}, \qquad \text{or equivalently} \qquad \Delta \geq E2^{b}. \tag{1}$$

For a fresh ciphertext, E includes encoding and encryption noise. Thus meeting a precision target already entails a scale cost to overcome this noise before any homomorphic evaluation. For standard CKKS ciphertext, the common scale Δ does not allow each slot precisions to be adjusted independently to match different requirements b_j.

When the scale is kept approximately constant at Δ, the initial modulus Q typically consists of the final output modulus Q_{out} multiplied by L factors of approximately Δ, i.e., $Q \approx Q_{\text{out}} \prod_{\ell=1}^{L} \Delta = Q_{\text{out}}\Delta^{L}$. Reducing Δ can therefore reduce the required modulus.

Existing methods exploit tradeoffs between modulus, computation, and storage. Numerical representations improve packing and arithmetic efficiency [CSV17, BBB+17, CIV18], while generalized plaintext spaces reduce noise growth [CLPX18, GV25]. Ciphertext decomposition controls key-switching noise and offers tradeoffs between modulus requirements, computation, and evaluation-key storage [BGV12, HK20]. Tuple-CKKS introduces a new ciphertext multiplication algorithm with lower modulus consumption [CCKS23]. Scale-optimization methods differ in their emphasis on precision, running time, and modulus consumption, but all improve computational efficiency by adjusting scales during evaluation [LHC+22, LCK+23, CCK+25].

Bootstrapping can produce an output ciphertext whose modulus Q is larger than that of the input ciphertext, but incurs additional computational and modulus costs [CHK+18]. Algorithms for homomorphic linear transformations reduce running time [HS14], while later improvements also reduce evaluation-key storage [HS18]. In CKKS bootstrapping, these transformations offer tradeoffs between running time and modulus consumption [CCS19, HHC19]. Further bootstrapping improvements increase output precision [LLL+21, LLK+22, BCC+22] or reduce modulus consumption [KPK+22, SSKM25, CKSS25, KCY25, YZC+26]. For small batches, specialized packing and bootstrapping algorithms reduce latency [CHK+18, Zhe26, CK25, CS25].

These methods improve representations, arithmetic, scale management, and bootstrapping, but all operate under the scale constraint $\Delta \geq E2^{b_{\max}}$, where $b_{\max} = \max_j b_j$. For $b_{\max} \geq 0$, this restricts scale selection to $\Delta \geq E$. We overcome this constraint by reallocating precision among independent values packed into a single ciphertext. The required scale is then governed by the

average precision requirement $\bar{b} = r^{-1} \sum_j b_j$ rather than the maximum $b_{\max}$. For negative $\bar{b}$, the target scale $E2^{\bar{b}}$ falls below E. In practical applications, reallocating precision from unused slots to useful slots can reduce the required ciphertext modulus effectively.

Specifically, **(i)** we enable arbitrary precision allocation across slots carrying independent messages within a single ciphertext, opening a new space of precision tradeoffs. **(ii)** We formulate precision allocation as an approximate closest-vector problem in the logarithmic lattice of cyclotomic units, and give an efficient, explicit Fourier method that constructs the corresponding public algebraic units with an explicit additive error bound. **(iii)** We exploit this allocation to reduce the ciphertext modulus Q using average rather than maximum precision requirements, derive allocation bounds under decryption and security constraints, and maintain the allocation through homomorphic evaluation.

1.1 Our Results and Techniques

To explain our mechanism, we abstract the polynomial represented by a ciphertext as $m+\eta$, where m is the message polynomial and η is the error polynomial. Both have real coefficients and are taken modulo $X^N + 1$, where N is a power of two. The $r = N/2$ slots hold $x_j = \sigma_j(m)$, where the embeddings σ_j evaluate polynomials at selected roots of $X^N + 1$. A nonzero public polynomial $p \in R = \mathbb{Z}[X]/(X^N + 1)$ can take different values $\sigma_j(p)$ across slots. We first divide m by p and later restore the message by multiplying by p. With error η in this representation, restoration gives

$$\frac{m}{p} + \eta \quad \xrightarrow{\times p} \quad m + p\eta.$$

The error in slot j is multiplied by $\sigma_j(p)$, so small $|\sigma_j(p)|$ increase its precision and large values decrease it, while the messages remain unchanged and independent.

From precision requirements to a lattice target. The weights $|\sigma_j(p)|$ must come from one integral polynomial. The algebraic norm of p is $\mathrm{Norm}(p) = \left(\prod_{j=0}^{r-1} |\sigma_j(p)|\right)^2$. A nonunit has norm greater than one, so precision gains in some slots come at a greater total precision loss in the others. It also need not be invertible modulo every ciphertext modulus. We therefore choose a unit $u \in R^\times$, whose inverse also lies in R and whose weights satisfy $\prod_j |\sigma_j(u)| = 1$. The vectors $(\log_2 |\sigma_j(u)|)_j$ form a lattice in the hyperplane $H = \{\mathbf{t} \in \mathbb{R}^r : \sum_j t_j = 0\}$. For precision requirements b_j, we target weights $w_j = 2^{\bar{b}-b_j}$, or equivalently logarithms $t_j = \bar{b} - b_j$. Since $\sum_j t_j = 0$, precision allocation becomes the problem of finding a lattice point close to $\mathbf{t} \in H$.

An explicit Fourier construction. Computing unit groups of general number fields is believed to be classically hard [EHKS14, STOC 2014], and the closest vector problem is NP-hard for general lattices [DKRS03]. We give an efficient, explicit approximation method using the structure of cyclotomic units. We choose a cyclotomic unit $u_\star$ whose conjugates cyclically shift its logarithmic weights. We choose their exponents to approximate the target by a sum of these shifted logarithmic weights. For a product of these conjugates, the logarithmic weights are a circular convolution of the exponent vector with $(\log_2 |\sigma_j(u_\star)|)_j$. We prove that the Fourier transform of this vector is nonzero at every nonzero frequency, making this convolution invertible on H. Fourier inversion therefore gives real exponents for any target in H; rounding them to integers produces a unit u satisfying

$$\max_j \left|\log_2 |\sigma_j(u)| - t_j\right| \le C_r, \qquad C_r := \frac{1}{2} \sum_{j=0}^{r-1} \left|\log_2 |\sigma_j(u_\star)|\right|. \tag{2}$$

This additive bound is independent of the target precision (Theorem 4.1). Additionally, for periodic precision requirements, construction in a subring makes the bound depend on the number of precision classes.

We maintain this representation through homomorphic evaluation and track the error introduced and propagated by each operation (Sections 7.1 and 7.2).

Modulus savings and allocation bounds. Under a fixed common error bound E, the target weights give $\Delta = E2^{\bar{b}}$, since $Ew_j/\Delta = 2^{-b_j}$ for every slot. For example, four equally sized classes requiring $(160, 32, 32, 32)$ bits have average 64, reducing the scale for the target weights from $E2^{160}$ to $E2^{64}$ even though all slots are useful. With one useful class and three unused classes, precision targets $(20, -20, -20, -20)$ give average -10 and scale $E2^{-10} < E$, also reducing the cost of overcoming encryption noise.

We analyze how noise growth depends on the amount of precision transferred. The amount of precision that can be reassigned is limited by both output accuracy and correct decryption. Useful slots must meet their precision requirements. Errors in all slots, including unused ones, also affect whether the ciphertext can be correctly decrypted. For a fixed circuit and parameter schedule with two equally sized slot groups, we compute a range of precision transfers for which our bounds guarantee output precision and correct decryption (Theorem 6.1).

1.2 Related Work

Concurrent work. Independent concurrent works by Cheon and Lee [CL26], Dumezy [Dum26], and Ma, Liu, Wang, and Wang [MLWW26] share with ours the use of public ring elements to redistribute error across slots, enabling tradeoffs between packing capacity, precision, and modulus consumption. A detailed comparison appears in Appendix F.

Other related work. Earlier encodings use roots of unity for complex arithmetic [CSV17], non-integral bases to reduce coefficient growth [BBB+17], and Laurent polynomials to increase packing capacity [CIV18]. At the level of plaintext spaces, CHIMERA [BGGJ20] enables switching between homomorphic encryption schemes through a common algebraic representation that includes algebraic plaintext quotients. CLPX [CLPX18] replaces the integer plaintext modulus by a polynomial, reducing noise growth for large-integer arithmetic. Related constructions extend this approach to complex arithmetic [BCIV20] and combine polynomial plaintext moduli with CKKS packing to reduce ciphertext storage [CIL21]. GBFV [GV25] combines such moduli with algebraic scaling, SIMD packing, and bootstrapping. The polynomial moduli in these schemes define the plaintext space. Our construction instead introduces a new capability: allocating precision across slots within a single CKKS ciphertext while retaining its plaintext space and common scalar scale.

High-precision integer methods represent each large value using smaller components. Nested residue number systems store residues of one integer in several slots of the same ciphertext [BK25]. Radix-based methods use digits and homomorphic carry processing [Kim25, CPL26], while dBFV uses plaintext decomposition to control noise for large plaintext moduli [PZZ26]. These components jointly represent one value; our construction reallocates precision among independent CKKS messages with different precision requirements. The two approaches do not conflict.

Organization. Section 2 gives the CKKS and algebraic background. Sections 3 and 4 develop the precision-allocation framework and the explicit Fourier construction of units. Sections 5 and 6 analyze scale and modulus savings and determine how much precision can be transferred while

meeting output precision and decryption requirements. Section 7 describes homomorphic evaluation and analyzes errors, security, and computation and storage costs. Section 8 presents the implementation and experiments, including a comparison with Tuple CKKS.

2 Preliminaries

2.1 Notations

Let ζ be a primitive root of unity of order $m_{\mathrm{cyc}} > 2$, and let $N := \varphi(m_{\mathrm{cyc}})$, with φ Euler's totient function, $K := \mathbb{Q}(\zeta)$, and $R := \mathbb{Z}[\zeta] \simeq \mathbb{Z}[X]/(\Phi_{m_{\mathrm{cyc}}}(X))$, where $\Phi_{m_{\mathrm{cyc}}}$ is the cyclotomic polynomial of order m_{cyc} and X is identified with ζ. The embeddings $\sigma : K \hookrightarrow \mathbb{C}$ are determined by $\sigma(\zeta) = \zeta^a$ for $1 \le a < m_{\mathrm{cyc}}$ with $\gcd(a, m_{\mathrm{cyc}}) = 1$. They form conjugate pairs $\sigma, \overline{\sigma}$, where $\overline{\sigma}(z) = \overline{\sigma(z)}$. Since $|\overline{\sigma}(z)| = |\sigma(z)|$, choose one representative σ_j from each pair and set $r := N/2$. Extending these maps to polynomials with real coefficients, write $\sigma : \mathbb{R}[X]/(\Phi_{m_{\mathrm{cyc}}}(X)) \longrightarrow \mathbb{C}^r, f \longmapsto (\sigma_j(f))_{j=0}^{r-1}$. The inverse $\sigma^{-1}(\mathbf{z})$ denotes the unique polynomial of degree less than N with real coefficients whose selected embeddings equal $\mathbf{z}$.

For a nonnegative integer n, write $[n] := \{0, \dots, n\}$. For an odd integer $Q > 1$, put $R_Q := R/QR$. The notation $[f]_Q$ means reduction of each coefficient to $(-Q/2, Q/2)$; we also use this representative for an element of R_Q. The operation $\lfloor f \rceil$ rounds each real coefficient to the nearest integer, with a fixed rule for ties. Vector products and quotients are taken coordinate by coordinate.

For a vector $\mathbf{z}$, write $\|\mathbf{z}\|_\infty := \max_j |z_j|$. For $f = \sum_{k=0}^{N-1} f_k X^k$ with real coefficients, define $\|f\|_{\mathrm{coef},\infty} := \max_k |f_k|$ and $\|f\|_{\sigma,\infty} := \max_j |\sigma_j(f)|$. All logarithms are to base two unless stated otherwise.

2.2 CKKS Ciphertexts and Precision

CKKS [CKKS17] uses $m_{\mathrm{cyc}} = 2N$ with N a power of two, so $R = \mathbb{Z}[X]/(X^N + 1)$. The $r = N/2$ coordinates of $\sigma(f)$ are called slots. CKKS encodes a vector $\mathbf{x} \in \mathbb{C}^r$ at a real scale $\Delta > 0$ as

$$\mu = \lfloor \Delta \sigma^{-1}(\mathbf{x}) \rceil \in R. \tag{3}$$

The encoding error $e_{\mathrm{enc}} := \mu - \Delta\sigma^{-1}(\mathbf{x})$ satisfies $\|e_{\mathrm{enc}}\|_{\mathrm{coef},\infty} \le 1/2$. Let $s \in R$ be the secret key and pk its public key. Encrypting $[\mu]_Q$ gives a ciphertext $\mathsf{ct} = (c_0, c_1) \in R_Q^2$. Let $e_{\mathrm{dec}} \in R$ denote the decryption error arising from encryption. The ciphertext phase $c_0 + c_1 s$ satisfies $c_0 + c_1 s \equiv \mu + e_{\mathrm{dec}} \pmod{Q}$.

Decryption computes $\mathsf{Dec}_s(\mathsf{ct}) := [c_0 + c_1 s]_Q$. If $\|\mu + e_{\mathrm{dec}}\|_{\mathrm{coef},\infty} < Q/2$, decryption returns $\mu + e_{\mathrm{dec}}$, so $e_{\mathrm{dec}} = \mathsf{Dec}_s(\mathsf{ct}) - \mu$. Decoding then gives $\widehat{x}_j := \sigma_j(\mathsf{Dec}_s(\mathsf{ct}))/\Delta = x_j + \sigma_j(e_{\mathrm{enc}} + e_{\mathrm{dec}})/\Delta$. A target of b_j bits of absolute precision requires $|\widehat{x}_j - x_j| \le 2^{-b_j}$.

For $f, g \in \mathbb{R}[X]/(X^N + 1)$, set $\Gamma_N := \csc(\pi/(2N))$. The coefficient and embedding norms satisfy

$$\|f\|_{\sigma,\infty} \le \Gamma_N \|f\|_{\mathrm{coef},\infty}, \qquad \|f\|_{\mathrm{coef},\infty} \le \frac{1}{r} \sum_{j=0}^{r-1} |\sigma_j(f)| \le \|f\|_{\sigma,\infty}. \tag{4}$$

Appendix A proves (4) and the optimality of Γ_N. In particular, coefficient rounding gives $\|e_{\mathrm{enc}}\|_{\sigma,\infty} \le \Gamma_N/2$. Multiplication also satisfies $\|fg\|_{\sigma,\infty} \le \|f\|_{\sigma,\infty} \|g\|_{\sigma,\infty}$.

2.3 Units and Logarithmic Embeddings

Let $R^\times = \{u \in R : \exists v \in R,\ uv = 1\}$ denote the group of units. Dirichlet's unit theorem states that a number field with r_1 real embeddings and r_2 pairs of nonreal complex embeddings has $r_1 + r_2 - 1$

fundamental units. Here $r_1 = 0$ and $r_2 = r$, giving $r_1 + r_2 - 1 = r - 1$. Thus every unit $u \in R^\times$ can be written as $u = \pm\zeta^k \prod_{i=1}^{r-1} u_i^{n_i}$, where $k, n_i \in \mathbb{Z}$ and $u_1, \ldots, u_{r-1}$ are fixed multiplicatively independent units. Since $|\sigma_j(\pm\zeta^k)| = 1$, we have $|\sigma_j(u)| = \prod_{i=1}^{r-1} |\sigma_j(u_i)|^{n_i}$.

For nonzero $z \in K$, define

$$\mathrm{Log}(z) = (\log_2 |\sigma_j(z)|)_{j=0}^{r-1}, \qquad H = \Big\{\mathbf{t} \in \mathbb{R}^r : \sum_{j=0}^{r-1} t_j = 0\Big\}.$$

For a unit u, the norm identity $\prod_j |\sigma_j(u)|^2 = \mathrm{Norm}_{K/\mathbb{Q}}(u) = 1$ places $\mathrm{Log}(u)$ in H. Products become sums, and roots of unity have logarithm zero. Under this embedding, the unit group maps to a lattice of rank $r-1$ in H: $\Lambda = \mathrm{Log}(R^\times) = \Big\{\sum_{i=1}^{r-1} n_i \mathrm{Log}(u_i) : n_i \in \mathbb{Z}\Big\}$. The vectors $\mathrm{Log}(u_i)$ form a basis of H over $\mathbb{R}$, and their integer combinations are exactly the logarithms of units. For targets $w_j > 0$ with $\prod_j w_j = 1$, we seek a lattice point close to $(\log_2 w_j)_j \in H$ to obtain a unit u with $|\sigma_j(u)| \approx w_j$.

The cyclotomic unit group is $C_{m_{\mathrm{cyc}}} = R^\times \cap \langle -1, \zeta, 1 - \zeta^a : 1 \le a < m_{\mathrm{cyc}} \rangle$, where the generated group is taken in $K^\times$. For integers a coprime to m_{cyc}, put $c_a = (1-\zeta^a)/(1-\zeta)$. Since $c_{-a} = -\zeta^{-a} c_a$, we have $\mathrm{Log}(c_{-a}) = \mathrm{Log}(c_a)$. To avoid repetition, use $G = (\mathbb{Z}/m_{\mathrm{cyc}}\mathbb{Z})^\times / \{\pm 1\}$, which groups the indices a and $-a$ modulo m_{cyc} corresponding to conjugate embeddings. We identify G with a set of integer representatives containing 1. We use the following two classical results.

Lemma 2.1 ([Was97, Lemma 8.1])**.** *If $m_{\mathrm{cyc}} > 2$ is a prime power, then $C_{m_{\mathrm{cyc}}}$ is generated by -1, ζ, and c_a for $a \in G \setminus \{1\}$.*

Lemma 2.2 ([CDPR16, Theorem 2.8])**.** *If $m_{\mathrm{cyc}} > 2$ is a prime power, then $[\Lambda : \mathrm{Log}(C_{m_{\mathrm{cyc}}})] = h^+(m_{\mathrm{cyc}})$, where $h^+(m_{\mathrm{cyc}})$ is the class number of $\mathbb{Q}(\zeta + \zeta^{-1})$.*

For CKKS, $m_{\mathrm{cyc}} = 2N = 4r$ is a power of two. We construct an explicit basis.

Corollary 2.1 (An explicit basis)**.** *If $m_{\mathrm{cyc}} = 4r$ with $r \ge 2$ a power of two, then the vectors $\mathrm{Log}(c_{5^h})$, $1 \le h < r$, form a basis of H over $\mathbb{R}$.*

Proof. Since 5 has order r modulo $4r$ and $5^h \equiv 1 \pmod 4$ for $0 \le h < r$, the elements $5^0, 5^1, \ldots, 5^{r-1}$ represent G. By Lemma 2.1, the stated $r-1$ vectors generate $\mathrm{Log}(C_{m_{\mathrm{cyc}}})$ over $\mathbb{Z}$, since roots of unity have logarithm zero. Lemma 2.2 shows that $\mathrm{Log}(C_{m_{\mathrm{cyc}}})$ has finite index in Λ and hence has the same real span, namely H. Since $\dim H = r - 1$, these $r-1$ generators form a basis of H. □

3 Precision Allocation via Ring Multipliers

3.1 Encoding and Message Recovery

We use a public ring multiplier, namely a nonzero element p of the plaintext ring R, to allocate errors across slots. All slots remain available for independent messages, with none reserved for auxiliary data.

For a message x_j in slot j, we encode $x_j/\sigma_j(p)$ rather than x_j. If this value acquires an additive error η_j, restoration gives

$$\sigma_j(p)\left(\frac{x_j}{\sigma_j(p)} + \eta_j\right) = x_j + \sigma_j(p)\eta_j.$$

Let $B > 0$ bound $|\eta_j|$ in every slot before restoration, including accumulated encoding, decryption, and homomorphic evaluation errors. Restoration reduces this bound when $|\sigma_j(p)| < 1$ and increases it when $|\sigma_j(p)| > 1$, thus allocating the precision. For target errors $\boldsymbol{\varepsilon} = (\varepsilon_0, \ldots, \varepsilon_{r-1})$, where

$\varepsilon_j = 2^{-b_j}$ specifies b_j bits of absolute precision, we therefore seek p satisfying $|\sigma_j(p)|B \le \varepsilon_j$ for every j.

Specifically, to encrypt a message vector $\mathbf{x} = (x_0, \ldots, x_{r-1}) \in \mathbb{C}^r$ at modulus Q and scale $\Delta > 0$, we use the standard CKKS encoder to compute

$$\mu = \left\lfloor \Delta\, \sigma^{-1}\left(\frac{x_0}{\sigma_0(p)}, \ldots, \frac{x_{r-1}}{\sigma_{r-1}(p)}\right)\right\rceil \in R.$$

The positive real scale Δ is shared by all slots, but the absolute value of the scaling factor in slot j is $\Delta/|\sigma_j(p)|$.

1. *Encryption.* Compute $\mathsf{ct} = (c_0, c_1) = \mathsf{Enc}_{pk}([\mu]_Q)$ using standard CKKS encryption.

2. *Message recovery.* First compute the ciphertext $p\mathsf{ct} = ([pc_0]_Q, [pc_1]_Q)$. Then $\mathsf{Dec}_s(p\mathsf{ct}) = [p(c_0 + c_1 s)]_Q$ and $\widehat{x}_j = \sigma_j(\mathsf{Dec}_s(p\mathsf{ct}))/\Delta$ for each slot j.

Multiplication by p uses its integer coefficients directly and requires no additional plaintext scale.

To establish correctness, let e_{enc} be μ minus its unrounded value. Standard CKKS encryption satisfies $c_0 + c_1 s \equiv \mu + e_{\mathrm{dec}} \pmod{Q}$, where $e_{\mathrm{dec}} \in R$ is the decryption error. For this encoding, $\eta_j = \sigma_j(e_{\mathrm{enc}} + e_{\mathrm{dec}})/\Delta$. These definitions give

$$p(\mu + e_{\mathrm{dec}}) = \Delta\, \sigma^{-1}(\mathbf{x}) + p(e_{\mathrm{enc}} + e_{\mathrm{dec}}). \tag{5}$$

Multiplication by p cancels the division by its embeddings in the message term, but also multiplies the errors by p.

Applying (4) to the unrounded encoding and using the coefficient rounding bound $1/2$ gives $\|\mu\|_{\mathrm{coef},\infty} \ge \Delta|x_j|/(\Gamma_N|\sigma_j(p)|) - 1/2$ for each slot j. Even if some coefficients of μ lie outside $(-Q/2, Q/2)$, Lemma 3.1 gives a sufficient condition for correct message recovery after reduction modulo Q. Its proof is given in Appendix A.

Lemma 3.1 (Correctness of modular restoration)**.** *If* $\Delta\|\sigma^{-1}(\mathbf{x})\|_{\mathrm{coef},\infty} + \|p(e_{\mathrm{enc}} + e_{\mathrm{dec}})\|_{\mathrm{coef},\infty} < \frac{Q}{2}$, *then* $\mathsf{Dec}_s(p\mathsf{ct}) = p(\mu + e_{\mathrm{dec}})$, *and, for each slot* j, $\widehat{x}_j = x_j + \frac{\sigma_j(p)\sigma_j(e_{\mathrm{enc}} + e_{\mathrm{dec}})}{\Delta}$.

The following proposition gives finer control over p by bounding its embeddings. The proof is given in Appendix A.

Proposition 3.1 (Multiplier bounds for target precisions)**.** *If* $|\sigma_j(e_{\mathrm{enc}} + e_{\mathrm{dec}})|/\Delta \le B_j$ *and* $|\sigma_j(p)| \le \varepsilon_j/B_j$ *for every slot* j, *with possibly different bounds* $B_j > 0$, *then* $\|p(e_{\mathrm{enc}} + e_{\mathrm{dec}})\|_{\mathrm{coef},\infty} \le (\Delta/r)\sum_{j=0}^{r-1}\varepsilon_j$. *If also*

$$\Delta\Big(\|\sigma^{-1}(\mathbf{x})\|_{\mathrm{coef},\infty} + \frac{1}{r}\sum\nolimits_{j=0}^{r-1}\varepsilon_j\Big) < \frac{Q}{2}, \tag{6}$$

then $|\widehat{x}_j - x_j| \le \varepsilon_j$ *for every slot* j.

Together, Lemma 3.1 and Proposition 3.1 give sufficient conditions for recovering the messages at the target precisions. The recovery condition requires the coefficients of $p(\mu + e_{\mathrm{dec}})$ to lie in $(-Q/2, Q/2)$. Those of $\mu + e_{\mathrm{dec}}$ before restoration may lie outside it.

The same recovery criterion applies after homomorphic evaluation. For an output ciphertext $\mathsf{ct}_{\mathrm{out}} = (c_0, c_1)$ at modulus Q_{out} and scale Δ_{out}, let $M_{\mathrm{out}} \in R$ satisfy $p(c_0 + c_1 s) \equiv M_{\mathrm{out}} \pmod{Q_{\mathrm{out}}}$. For the intended output $\mathbf{x}_{\mathrm{out}}$, write

$$\frac{\sigma_j(M_{\mathrm{out}})}{\Delta_{\mathrm{out}}} = x_{\mathrm{out},j} + \sigma_j(p)\eta_{\mathrm{out},j}, \qquad |\eta_{\mathrm{out},j}| \le B_{\mathrm{out},j}.$$

Here $\eta_{\mathrm{out},j}$ is the error after division by the scale and before restoration, bounded by $B_{\mathrm{out},j} > 0$. The common output bound is $B_{\mathrm{out}} := \max_{0 \le j < r} B_{\mathrm{out},j}$. Section 7.1 shows how to maintain this representation, and Section 7.2 derives the accumulated error bounds, which may depend on p and the computation. Recovery at the required precisions is guaranteed by:

$$\begin{cases} |\sigma_j(p)| B_{\mathrm{out},j} \le \varepsilon_j, & \text{for each required output slot } j, \\ \dfrac{\|M_{\mathrm{out}}\|_{\mathrm{coef},\infty}}{\Delta_{\mathrm{out}}} \le \|\sigma^{-1}(\mathbf{x}_{\mathrm{out}})\|_{\mathrm{coef},\infty} + \dfrac{1}{r} \sum_{j=0}^{r-1} |\sigma_j(p)| B_{\mathrm{out},j} < \dfrac{Q_{\mathrm{out}}}{2\Delta_{\mathrm{out}}}. \end{cases} \tag{7}$$

The second condition includes all slots, even those whose outputs are not required. As in Proposition 3.1, these conditions ensure that centered decryption of $p\mathsf{ct}_{\mathrm{out}}$ returns M_{out}, giving $|\widehat{x}_{\mathrm{out},j} - x_{\mathrm{out},j}| \le |\sigma_j(p)| B_{\mathrm{out},j} \le \varepsilon_j$ for each required output slot.

Rewriting the same encoding with a smaller scale does not itself reduce costs. For any $\lambda > 0$, the encoding can be written using scale Δ/λ and values $x_j/(\sigma_j(p)/\lambda)$, since $(\Delta/\lambda)\, x_j/(\sigma_j(p)/\lambda) = \Delta x_j/\sigma_j(p)$. With the same rounding, both forms give the same polynomial μ. For fixed p and an unchanged evaluation circuit, corresponding executions use the same modulus chain and integer operations. Proposition A.1 in Appendix A gives the formal statement and proof.

4 Construction of Ring Multipliers

4.1 From Precision Requirements to Embedding Weights

We determine target values for $|\sigma_j(p)|$ from precision targets $\mathbf{b} = (b_j)_{j=0}^{r-1} \in \mathbb{R}^r$. Unused slots may receive negative precision targets. With $\varepsilon_j := 2^{-b_j}$, the precision condition in (7) is $|\sigma_j(p)| B_{\mathrm{out},j} \le \varepsilon_j$.

For any $p \in R \setminus \{0\}$, $\prod_{j=0}^{r-1} |\sigma_j(p)|^2 = \mathrm{Norm}_{K/\mathbb{Q}}(p) \ge 1$, with equality if and only if $p \in R^\times$. Thus $|\sigma_j(p)| < 1$ cannot hold for every j. For a unit $u \in R^\times$, $|\sigma_j(u)| < 1$ in some slots requires $|\sigma_j(u)| > 1$ in others, with $\prod_j |\sigma_j(u)| = 1$. If p is not a unit, then $\prod_j |\sigma_j(p)| > 1$. Thus the same reductions in some slots require greater overall compensation from the others than for a unit. We therefore choose a unit $u \in R^\times$ as the multiplier p.

Set $\bar{b} := r^{-1} \sum_{j=0}^{r-1} b_j$. Choose target weights $w_j \propto \varepsilon_j$ for $|\sigma_j(u)|$. Here w_j specifies the desired factor by which restoration scales the error in slot j. Normalizing them so that $\prod_j w_j = 1$ gives

$$w_j := 2^{\bar{b} - b_j}, \qquad t_j := \log_2 w_j = \bar{b} - b_j.$$

If $B_{\mathrm{out}} \le 2^{-\bar{b}}$, the ideal error bounds satisfy $B_{\mathrm{out}} w_j \le 2^{-b_j}$. For example, suppose half the slots require 60 bits of precision and the other half require 20 bits. Then $\bar{b} = 40$, giving logarithmic targets $(-20, 20)$ and weights $(2^{-20}, 2^{20})$, which at $B_{\mathrm{out}} = 2^{-40}$ yield ideal error bounds $(2^{-60}, 2^{-20})$.

Set the target vector of logarithmic weights $\mathbf{t} := (t_j)_{j=0}^{r-1}$. Since $\sum_j t_j = 0$, we have $\mathbf{t} \in H$ and seek $u \in R^\times$ with $\mathrm{Log}(u) \approx \mathbf{t}$. The lattice Λ is discrete, so exact equality need not be possible. The next subsection gives a construction and bounds its deviation from $\mathbf{t}$.

4.2 Explicit Construction via Fourier Inversion

We construct $u \in R^\times$ from the target $\mathbf{t} \in H$. Let $r \ge 2$ be a power of two and $\zeta := e^{2\pi i/(4r)}$. Define the automorphism $\tau : K \to K$ by $\tau(\zeta) = \zeta^5$. Use the slot order $\sigma_j(\zeta) = \zeta^{5^j}$, $0 \le j < r$, and take subscripts modulo r. Choose the unit

$$u_\star := \zeta^{-2} c_5 = \zeta^{-2} + \zeta^{-1} + 1 + \zeta + \zeta^2.$$

The symmetric expression makes every $\sigma_j(u_\star)$ real. Since ζ^{-2} is a root of unity, $\mathrm{Log}(u_\star) = \mathrm{Log}(c_5)$. We choose c_5 to match the slot order $\sigma_j(\zeta) = \zeta^{5^j}$, and its numerator at slot j equals its denominator at slot $j+1$.

Set $\ell_j := \log_2 |\sigma_j(u_\star)|$. Since $\sigma_j(\tau^{-k}(u_\star)) = \sigma_{j-k}(u_\star)$, products of conjugates give the circular convolution

$$u(\mathbf{n}) := \prod_{k=0}^{r-1} \tau^{-k}(u_\star)^{n_k}, \quad \mathbf{n} \in \mathbb{Z}^r, \quad \log_2 |\sigma_j(u(\mathbf{n}))| = \sum_{k=0}^{r-1} \ell_{j-k} n_k. \tag{8}$$

Negative exponents are allowed because every factor is a unit. We use the Fourier transform to solve the convolution equation. For any vector $\mathbf{z} = (z_j)_{j=0}^{r-1} \in \mathbb{C}^r$, define $\widehat{z}_k := \sum_{j=0}^{r-1} z_j e^{-2\pi i jk/r}$, with inverse $z_j = \frac{1}{r} \sum_{k=0}^{r-1} \widehat{z}_k e^{2\pi i jk/r}$. The following lemma ensures that the convolution can realize every target in H with real coefficients.

Lemma 4.1 (Spanning by logarithms of conjugates)**.** *The vectors* $\mathrm{Log}(\tau^k(u_\star))$, $0 \le k < r$, *span* H *over* $\mathbb{R}$. *Consequently,* $\widehat{\ell}_0 = 0$ *and* $\widehat{\ell}_k \neq 0$ *for* $1 \le k < r$.

Proof. For $1 \le h < r$, $\prod_{k=0}^{h-1} \tau^k(u_\star) = \zeta^{(1-5^h)/2} c_{5^h}$, hence $\sum_{k=0}^{h-1} \mathrm{Log}(\tau^k(u_\star)) = \mathrm{Log}(c_{5^h})$. Corollary 2.1 and $\tau^k(u_\star) \in R^\times$ therefore give $\mathrm{span}_{\mathbb{R}}\{\mathrm{Log}(\tau^k(u_\star)) : 0 \le k < r\} = H$.

The convolution matrix $\mathbf{L} = (\ell_{j-k})_{0 \le j,k < r}$ has columns $\mathrm{Log}(\tau^{-k}(u_\star))$. Since $\tau^r = \mathrm{id}_K$, these span H, so $\mathrm{rank}\, \mathbf{L} = r-1$. Fourier diagonalization therefore shows that exactly $r-1$ of $\widehat{\ell}_0, \ldots, \widehat{\ell}_{r-1}$ are nonzero. Since $u_\star$ is a unit, $\widehat{\ell}_0 = \sum_j \ell_j = \log_2 \prod_j |\sigma_j(u_\star)| = 0$. Thus $\widehat{\ell}_k \neq 0$ for $1 \le k < r$. □

Theorem 4.1 (Construction and rounding bound)**.** *For* $\mathbf{t} \in H$, *take the inverse Fourier transform* $\mathbf{a} \in \mathbb{R}^r$ *of* $\widehat{a}_0 := 0$ *and* $\widehat{a}_k := \widehat{t}_k / \widehat{\ell}_k$ *for* $1 \le k < r$. *Set* $n_j := \lfloor a_j \rceil$ *and form* $u(\mathbf{n})$ *as in* (8). *Then* $u(\mathbf{n}) \in R^\times$ *and*

$$\| \mathrm{Log}(u(\mathbf{n})) - \mathbf{t} \|_\infty \le \frac{1}{2} \sum_{j=0}^{r-1} |\ell_j|.$$

Proof. Conjugate symmetry of the transforms of the real vectors $(t_j)_j$ and $(\ell_j)_j$ makes $\mathbf{a}$ real. Since $\widehat{t}_0 = 0$, Lemma 4.1 and Fourier inversion give $\sum_k \ell_{j-k} a_k = t_j$. Using $|n_k - a_k| \le 1/2$, we obtain $|\log_2 |\sigma_j(u(\mathbf{n}))| - t_j| = |\sum_k \ell_{j-k}(n_k - a_k)| \le \sum_k |\ell_{j-k}|\, |n_k - a_k| \le \frac{1}{2} \sum_{k=0}^{r-1} |\ell_k|$. The product defining $u(\mathbf{n})$ is a unit. □

The bound gives $2^{t_j - \frac{1}{2}\sum_{k=0}^{r-1} |\ell_k|} \le |\sigma_j(u(\mathbf{n}))| \le 2^{t_j + \frac{1}{2}\sum_{k=0}^{r-1} |\ell_k|}$. It measures the deviation of the weights from their targets.

For periodic targets $t_{j+m} = t_j$, where $m \ge 2$ divides r, apply the same construction in $\mathbb{Z}[\zeta_{4m}]$ with $\zeta_{4m} := \zeta^{r/m}$. Use $(t_0, \ldots, t_{m-1})$ as the target, replacing r and ζ by m and ζ_{4m} in the construction and its bound. This target also has coordinate sum zero. The resulting unit satisfies $\sigma_{j+m}(u) = \sigma_j(u)$, giving m groups of r/m slots with deviation bound $\frac{1}{2} \sum_{j=0}^{m-1} |\ell_j|$, using the subfield values of ℓ_j. Slots in each group retain independent messages.

For example, with $m = 2$, set $h := \zeta_8 + \zeta_8^{-1}$. Then $h^2 = 2$, and the chosen unit is $u_\star = 1 + h$, with $u_\star^{-1} = h - 1$. Since $\sigma_j(h) = (-1)^j \sqrt{2}$, the multiplier $u = u_\star^k$ has weights (α^k, α^{-k}) on the even and odd slots, where $\alpha := 1 + \sqrt{2}$. For a target (t_0, t_1) with $t_1 = -t_0$, choose $k := \lfloor t_0 / \log_2 \alpha \rceil$. Rounding this single exponent gives the tighter deviation bound $\frac{1}{2} \log_2 \alpha \approx 0.636$ bits for either logarithmic weight. Notably, this bound is independent of the amount of precision transferred.

5 From Precision Allocation to Resource Savings

We now quantify how precision allocation can reduce the scales and ciphertext moduli needed at the required precision. Computation and storage costs are analyzed in Section 7.3.

We view the computation as a directed acyclic graph, with nodes for input ciphertexts and homomorphic operations and edges for data dependencies. At node v, let Q_v, Δ_v be the ciphertext modulus and scale, and let $x_{v,j}$ be the intended value in slot j, for $0 \le j < r$. Since the absolute values of messages affect error growth and recovery, we use bounds $|x_{v,j}| \le A_{v,j}$, starting from bounds on the inputs.

Ordinary CKKS ($p = 1$) and our construction ($p = u$) choose their own N, Δ_v, Q_v and auxiliary key-switching modulus P for the same logical circuit, input bounds $A_{v,j}$, precision targets 2^{-b_j}, and number of useful inputs. Both use the same security target, secret and error distributions, and failure probability bound. Superscripts o, u distinguish ordinary CKKS from our method throughout the comparisons.

5.1 Scale and Modulus Reduction

For the scale and modulus comparison, we further fix the same N for both methods. We first relate the required scale to the average precision, then compare initial moduli subject to the precision and recovery conditions.

Scale reduction. Assume that all slots have precision targets 2^{-b_j}, with $\bar{b}$, w_j, and t_j as in Section 4.1. Suppose the normalized error η_j in each slot before restoration satisfies $|\eta_j| \le E/\Delta$, where $E > 0$ bounds the error before division by the scale and is fixed for all weights and scales being compared. For a fresh ciphertext at input node v, one can take $E = B_{\mathrm{enc},v} + B_{\mathrm{dec},v}$, where $B_{\mathrm{enc},v}$ and $B_{\mathrm{dec},v}$ bound, respectively, the encoding and encryption errors in each embedding before division by the scale.

Proposition 5.1 (Scale and average precision)**.** *Under the common bound above, allow positive embedding weights ω_j with $\prod_j \omega_j = 1$. The least scale for which the restored error bounds meet all precision targets is*

$$\min_{\substack{\omega_j > 0 \\ \prod_j \omega_j = 1}} E \max_j (2^{b_j} \omega_j) = E 2^{\bar{b}}. \tag{9}$$

The target weights $\omega_j = w_j = 2^{\bar{b} - b_j}$ attain this minimum.

Proof. For fixed weights, the conditions $E\omega_j/\Delta \le 2^{-b_j}$ are equivalent to $\Delta \ge E \max_j (2^{b_j}\omega_j)$. Since $\prod_j (2^{b_j}\omega_j) = 2^{r\bar{b}}$, the maximum is at least $2^{\bar{b}}$. The target weights make every term equal to this value. □

For an actual unit u, define $\delta_u := \max_j(\log_2 |\sigma_j(u)| - t_j)$, its deviation above the logarithmic targets. Under the same common bound, the minimum scales required by the error bounds for our method and ordinary CKKS are

$$\Delta_{\mathrm{unit}} = E 2^{\bar{b} + \delta_u}, \qquad \Delta_{\mathrm{std}} = E 2^{b_{\max}}.$$

Thus the scale decreases by $b_{\max} - \bar{b} - \delta_u$ bits when this quantity is positive. Theorem 4.1 bounds this deviation for the constructed unit.

A negative b_j allows an absolute error of up to $2^{-b_j} > 1$ in slot j. For $b_{\max} > 0$, ordinary CKKS requires $\Delta_{\mathrm{std}} = E 2^{b_{\max}} > E$ under the common bound above. The average $\bar{b}$ can nevertheless be

negative. If $\bar{b} < 0$, the target weights in Proposition 5.1 give a scale $E2^{\bar{b}} < E$. For an actual unit, if $\bar{b} + \delta_u < 0$, then $\Delta_{\text{unit}} < E < \Delta_{\text{std}}$. Hence our method can use a scale below E while its restored error bounds still meet all precision targets, including those in slots requiring high precision.

Modulus reduction. Compare evaluations of the same circuit by ordinary CKKS and our method, both satisfying the precision and recovery conditions in (7), with accumulated error bounds as in Section 7.2. Define $\gamma_v := \Delta_v^{\text{o}}/\Delta_v^{\text{u}}$, and let D_v be the divisor when rescaling is performed at node v. For a multiplication node v with inputs a, b, (19) relates the divisor to both input and output scales through $D_v^{\text{u}}/D_v^{\text{o}} = \gamma_v/(\gamma_a\gamma_b)$. Assume the corresponding modulus chains have L rescalings and no other modulus reductions. Let Q_t be the modulus after step t, starting from Q_0, and D_t the divisor at that step. Since $Q_{t-1} = D_t Q_t$, the saving in $\log_2 Q_0$ is

$$G_Q := \log_2 \frac{Q_0^{\text{o}}}{Q_0^{\text{u}}} = \log_2 \frac{Q_L^{\text{o}}}{Q_L^{\text{u}}} + \sum_{t=1}^{L} \log_2 \frac{D_t^{\text{o}}}{D_t^{\text{u}}}. \tag{10}$$

For $L \geq 1$ successive squarings, let Δ_t be the scale after step t, starting from Δ_0, and set $\gamma_t := \Delta_t^{\text{o}}/\Delta_t^{\text{u}}$. The updates $D_t = \Delta_{t-1}^2/\Delta_t$ give $Q_0 = (Q_L/\Delta_L)\Delta_0^2 \prod_{t=1}^{L-1} \Delta_t$, hence

$$G_Q = 2\log_2 \gamma_0 + \sum_{t=1}^{L-1} \log_2 \gamma_t - \log_2 \frac{Q_L^{\text{u}}/\Delta_L^{\text{u}}}{Q_L^{\text{o}}/\Delta_L^{\text{o}}}. \tag{11}$$

The first two terms give savings from smaller scales; a larger final Q_L/Δ_L for our method reduces these savings. The initial modulus is smaller exactly when $G_Q > 0$. These comparisons use the actual scales and moduli determined by the RNS primes. Section 7.3 compares computation and storage using the selected prime counts, and Section 6 next determines allocation ranges satisfying the precision and recovery conditions.

6 Precision Allocation under Recovery Constraints

The savings in Section 5 require correct recovery as well as the requested output precisions. Recovery of M_{out} depends on errors in every slot, including unused ones. We use (7) to determine how far precision can be redistributed while guaranteeing recovery.

6.1 Effects of Precision Differences on Error

Fix two equal groups of slots and write their weights as t^{s_j}, where $s_j \in \{-1, 1\}$ and $t = 2^\theta \geq 1$. Using the common output bound B_{out}, the restored error bounds are B_{out}/t and $B_{\text{out}}t$. Their precision bounds differ by 2θ bits, and the coefficient error contribution in (7) is bounded by $B_{\text{out}}(t + t^{-1})/2$.

Proposition 6.1 (Allocation bound for two levels)**.** *Set* $C := \frac{2}{B_{\text{out}}}\left(\frac{Q_{\text{out}}}{2\Delta_{\text{out}}} - \|\sigma^{-1}(\mathbf{x}_{\text{out}})\|_{\text{coef},\infty}\right)$. *If* $C > 2$ *and*

$$0 \leq \theta < \log_2 \left(\frac{C + \sqrt{C^2 - 4}}{2} \right), \tag{12}$$

then centered decryption of $u\mathsf{ct}_{\text{out}}$ *returns the entire polynomial* M_{out}.

The proof is given in Appendix A. Since the propagated bound B_{out} may depend on t, this proposition tests a chosen allocation. We now retain t as a variable to compute the range allowed by the propagated error bounds.

We use the node and slot notation of Section 5. At each node v, let $\eta_{v,j}$ be the normalized error in slot j before restoration, and choose a bound $|\eta_{v,j}| \le B_{v,j}$, extending the output notation of Section 3. Fix the circuit, its public constants, input moduli, rescaling divisors D_v, and positive rational input and plaintext scales $\Delta_v, \Delta_{\mathrm{pt}}$. Propagate the moduli and scales by the rules in Section 7.1.

Choose rational, t-independent bounds on the absolute values of inputs and public constants, and on the errors introduced by each operation. In addition to $B_{\mathrm{enc},v}$ and $B_{\mathrm{dec},v}$, write K_v for the relinearization or key-switching phase-error bound and R_v/D_v for the rescaling phase-error bound. For public constants, $B_{+,v}$ bounds the encoding error of an addend in each embedding, and $\beta_{v,j}$ bounds a multiplier's encoding error in slot j after division by its plaintext scale. Equation (18) and Appendix B specify these bounds in detail. They must hold simultaneously for the actual operation inputs, including those after public corrections. Let $S \subseteq [r-1]$ index the required output slots, and choose rational error thresholds ε_j no greater than their requested tolerances.

With these fixed data, set $\omega_j = |\sigma_j(u)|$ and write the restored error bounds as $\mathcal{E}_{v,j} = \omega_j B_{v,j}$. We first rewrite the rules in Section 7.2 and Appendix B in these variables, so that weight ratios and inverses cancel. For example, a rotation from input node a to output node v that moves slot k to slot j gives $\mathcal{E}_{v,j} = \mathcal{E}_{a,k} + \omega_j K_v/\Delta_v$. Since the remaining weights enter through sums and products with nonnegative coefficients, replacing them by upper bounds $W_j(t) \ge \omega_j$ preserves the error bounds. Take $W_j(t) = t^{s_j}$ for exact two-level weights, or $W_j(t) = \Gamma t^{s_j}$ for the construction in Theorem 4.1, with a rational $\Gamma \ge 2^{\frac{1}{2}\sum_k |\ell_k|}$. Denote the resulting bounds by $\mathcal{E}_{v,j}(t)$. Each $W_j(t)$ is a rational multiple of t or t^{-1}; therefore, the recursive sums, products, and slot permutations make $\mathcal{E}_{v,j}(t)$ a Laurent polynomial with nonnegative rational coefficients. Set

$$F(t) := \frac{1}{r}\sum_{j=0}^{r-1} \mathcal{E}_{\mathrm{out},j}(t), \qquad F(t) \in \mathbb{Q}_{\ge 0}[t, t^{-1}], \tag{13}$$

$$T := \frac{Q_{\mathrm{out}}}{2\Delta_{\mathrm{out}}} - \frac{1}{r}\sum_{j=0}^{r-1} A_{\mathrm{out},j}. \tag{14}$$

Here $F(t)$ bounds the coefficient error divided by Δ_{out}, and T is the decryption limit minus the message bound. By (4) and (7), $F(t) < T$ suffices for recovery.

For a rational Laurent polynomial f, a rational threshold b, and $\triangleleft \in \{<, \le\}$, let $\mathrm{Solve}(f, b, \triangleleft)$ return the exact solution set of $f(t) \triangleleft b$ on $t \ge 1$. Compare constants directly; otherwise multiply by t^k for a sufficiently large integer $k \ge 0$ and use real-root isolation and sign determination for the resulting rational polynomial, retaining endpoint inclusion.

Theorem 6.1 (Allocation bound for a general circuit)**.** *Under the assumptions above, Algorithm 1 terminates and returns $I = \log_2 J$, where*

$$J = \{t \ge 1 : F(t) < T,\ \mathcal{E}_{\mathrm{out},j}(t) \le \varepsilon_j \text{ for } j \in S\}. \tag{15}$$

The set J is an interval, possibly empty, a singleton, or unbounded; its finite endpoints are real algebraic numbers. For $t \in J$, any unit satisfying $\omega_j \le W_j(t)$ guarantees recovery and the required precisions on the joint error event above.

Proof. Induction over the circuit gives valid restored error bounds, so (7) applies. For $f(t) = \sum_k c_k t^k$ with $c_k \ge 0$,

$$\frac{\mathrm{d}^2}{\mathrm{d}\theta^2} f(2^\theta) = (\ln 2)^2 \sum_k k^2 c_k 2^{k\theta} \ge 0.$$

Algorithm 1: Range of allocations under recovery constraints

Input: Fixed circuit and data; bounds $W_j(t)$; optional precision requirements ε_j for $j \in S$.
Output: Interval I with endpoint inclusion, possibly empty.

for each *node v in circuit order* **do**
  Propagate $A_{v,j}$ and update Q_v, Δ_v as in Section 7.1.
  Form $\mathcal{E}_{v,j}(t)$ by the rewritten error propagation rules.
Compute $F(t)$ and T from (13)–(14).
if $T \le 0$ **then return** $\varnothing$.
$J \leftarrow \mathrm{Solve}(F, T, <)$.
for each *required output slot $j \in S$* **do**
  $J \leftarrow J \cap \mathrm{Solve}(\mathcal{E}_{\mathrm{out},j}, \varepsilon_j, \le)$.
return $I = \{\log_2 t : t \in J\}$.

Thus the conditions define intervals in θ; their intersection remains an interval under $t = 2^\theta$. Real-root isolation, sign determination, and comparison of real algebraic endpoints terminate and give the exact set in (15). □

All slots contribute to F, including those without precision requirements; take $S = \varnothing$ if no such requirements are imposed. For several outputs, intersect their conditions. For exact two-level weights, the common bound $B_{\mathrm{out}}(t)$ gives $F(t) \le B_{\mathrm{out}}(t)(t+t^{-1})/2$. Thus the test in Proposition 6.1, evaluated with $C(t) = 2T/B_{\mathrm{out}}(t)$, implies $F(t) < T$.

Units are constructed as in Section 4.2. For the even and odd slots considered there, use the base unit $u_\star$ and its exact weights for $u_\star^k$, retaining $\mathcal{K} = \{k \in \mathbb{Z}_{\ge 0} : \alpha^k \in J\}$, where $\alpha = 1 + \sqrt{2}$ as defined there. The allocations are $\theta = k \log_2 \alpha$, using $u_\star^k$ or $u_\star^{-k}$ according to the group order. A nonempty J may contain no α^k, leaving no unit from this construction that meets the bounds. Outside J, these bounds give no guarantee of recovery or precision.

6.2 Allocation with Dummy Slots

When a task needs outputs at different precisions, both groups can produce useful results. If all useful outputs require high precision, we can instead fill some slots with dummy values and assign smaller weights to the useful slots. Here N is fixed, so dummy values occupy some of the existing $r = N/2$ slots. Even if a dummy output has ideal value $x_{\mathrm{out},j} = 0$, its actual embedding $\sigma_j(M_{\mathrm{out}}) = \Delta_{\mathrm{out}} \sigma_j(u) \eta_{\mathrm{out},j}$ can be nonzero and must be included in (7).

To choose target weights, let $\mathcal{I}, \mathcal{D}$ be nonempty sets of useful and dummy slots that partition the r slots. Fix their sizes, the useful targets t_i for $i \in \mathcal{I}$, and a common error bound B_{out}. The unit constraint requires $\sum_{j \in \mathcal{D}} t_j = -\sum_{i \in \mathcal{I}} t_i$. By convexity of 2^t, the dummy contribution $(B_{\mathrm{out}}/r) \sum_{j \in \mathcal{D}} 2^{t_j}$ is smallest when all dummy targets equal their average, namely $t_j = -\sum_{i \in \mathcal{I}} t_i / |\mathcal{D}|$. If every useful target is $t_i = -\theta |\mathcal{D}|/|\mathcal{I}|$, this gives $t_j = \theta$ on the dummy slots and hence two levels. Here θ is the precision transferred from each dummy slot. Equal group sizes recover the targets $-\theta, \theta$ above.

For independent computations, a batch is a set of inputs packed into slots and processed by one circuit evaluation. Dummy slots reduce the number of useful inputs per batch, so the same task may require more batches. With sufficient parallel resources, more batches need not increase latency, although they increase total work.

7 Scheme Analysis

We give the homomorphic operations and error bounds used above, prove security, and account for computation and storage costs.

7.1 Homomorphic Evaluation

We keep the multiplier p fixed throughout homomorphic evaluation. For a ciphertext $\mathsf{ct} = (c_0, c_1)$ at modulus Q and scale Δ, let $M \in R$ denote the polynomial representing the message after restoration. We maintain

$$p(c_0 + c_1 s) \equiv M \pmod{Q}, \qquad \sigma(M) = \Delta(\mathbf{x} + \mathbf{e}), \tag{16}$$

where $\mathbf{x}$ is the intended value and $\mathbf{e}$ is its error. For a fresh ciphertext, $M = p(\mu + e_{\mathrm{dec}})$, where μ is obtained by applying (3) to $(x_j/\sigma_j(p))_{j=0}^{r-1}$.

Evaluation uses standard CKKS ciphertexts, only with the message in slot j represented as $x_j/\sigma_j(p)$. Key switching $\mathsf{KS}_{s'\to s}$ uses a public evaluation key evk to convert a ciphertext under s' into one under s. It preserves the message up to added error and leaves the modulus and scale unchanged. Relinearization $\mathsf{Relin}_{\mathrm{evk}}$ uses key switching to reduce a three-component ciphertext to two components. Both operations require no modification. We next consider the other homomorphic operations. We write M_+ and $M_\times$ for the polynomials after addition and multiplication, respectively.

$\mathsf{Add}(\mathsf{ct}_a, \mathsf{ct}_b)$. Let $\mathsf{ct}_a = (a_0, a_1)$ and $\mathsf{ct}_b = (b_0, b_1)$ have the same modulus Q, scale Δ, and secret key. Return $\mathsf{ct}_+ = ([a_0 + b_0]_Q, [a_1 + b_1]_Q)$. The output has modulus Q, scale Δ, and polynomial $M_+ = M_a + M_b$. It represents $\mathbf{x}_a + \mathbf{x}_b$ with error $\mathbf{e}_a + \mathbf{e}_b$. Subtraction follows the same rule with a change of sign.

$\mathsf{AddPlain}(\mathsf{ct}, \mathbf{a})$. To add a public vector $\mathbf{a} \in \mathbb{C}^r$, we encode $a_i/\sigma_i(p)$ in each slot, since the stored value is $x_i/\sigma_i(p)$. Compute $\mu_+ = \lfloor \Delta\, \sigma^{-1}((a_i/\sigma_i(p))_{i\in[r-1]}) \rceil$. Return $\mathsf{ct}_+ = ([c_0 + \mu_+]_Q, c_1)$, since adding μ_+ to c_0 increases the decryption expression $c_0 + c_1 s$ by μ_+. The output retains modulus Q and scale Δ, with $M_+ = M + p\mu_+$. Up to the existing error and the error from encoding $\mathbf{a}$, it therefore represents the logical sum $\mathbf{x} + \mathbf{a}$.

$\mathsf{MultPlain}(\mathsf{ct}, \mathbf{a}; \Delta_{\mathrm{pt}})$. For a public vector $\mathbf{a}$ and a plaintext scale $\Delta_{\mathrm{pt}} > 0$, compute $\mu_\times = \lfloor \Delta_{\mathrm{pt}}\, \sigma^{-1}(\mathbf{a}) \rceil$ and return $\mathsf{ct}_\times = ([\mu_\times c_0]_Q, [\mu_\times c_1]_Q)$. Since $(x_j/\sigma_j(p)) \cdot a_j = (x_j a_j)/\sigma_j(p)$, the public vector is encoded without division by $\sigma_j(p)$. The output has modulus Q and scale $\Delta\Delta_{\mathrm{pt}}$, with $M_\times = \mu_\times M$. It therefore represents the logical product $(x_j a_j)_{j\in[r-1]}$.

$\mathsf{Mult}_{\mathrm{evk}}(\mathsf{ct}_a, \mathsf{ct}_b)$. Let $\mathsf{ct}_a = (a_0, a_1)$ and $\mathsf{ct}_b = (b_0, b_1)$ represent $\mathbf{x}_a$ and $\mathbf{x}_b$ with modulus Q, secret key s, and scales Δ_a and Δ_b. First form the usual product with components $d_0 = [a_0 b_0]_Q$, $d_1 = [a_0 b_1 + a_1 b_0]_Q$, and $d_2 = [a_1 b_1]_Q$. Multiply every component by p, and return

$$\mathsf{ct}_\times = \mathsf{Relin}_{\mathrm{evk}}([pd_0]_Q, [pd_1]_Q, [pd_2]_Q).$$

This correction gives $\sigma_j(p) \cdot (x_{a,j} x_{b,j}/\sigma_j(p)^2) = x_{a,j} x_{b,j}/\sigma_j(p)$ in slot j, preserving the required representation. The output has modulus Q and scale $\Delta_a \Delta_b$. If $\kappa \in R$ is the phase error added by relinearization, then its polynomial in (16) is $M_\times = M_a M_b + p\kappa$. Rescaling is a separate operation. The output therefore represents the logical product $(x_{a,j} x_{b,j})_j$.

$\mathsf{RS}_D(\mathsf{ct})$. Let $Q = DQ'$, where $D > 1$ is odd. The components c_i are already written as centered integer representatives modulo Q. Set $\rho_i = [c_i]_D$ and return $\mathsf{ct}' = (c'_0, c'_1)$, where $c'_i = [(c_i - \rho_i)/D]_{Q'}$

for $i \in \{0,1\}$. For each coefficient $a = Dk + b$ with centered residue b, we have $|b| < D/2$, so $(a-b)/D = k = \lfloor a/D \rceil$. Thus the formula divides each coefficient by D and rounds to the nearest integer. The output has modulus Q' and scale $\Delta' = \Delta/D$. The result is independent of the chosen representatives. Lemma 7.1 below shows that this standard rescaling procedure preserves our representation.

$\underline{\mathsf{ModReduce}_{Q'}(\mathsf{ct})}$. For a divisor Q' of Q, return $\mathsf{ct}' = ([c_0]_{Q'}, [c_1]_{Q'})$. The scale remains Δ, and the same polynomial M satisfies (16) modulo Q'. Recovery at this modulus requires $\|M\|_{\mathrm{coef},\infty} < Q'/2$.

$\underline{\mathsf{Auto}_\tau(\mathsf{ct})}$. For a ring automorphism τ, applying τ to $p(c_0 + c_1 s) \equiv M \pmod{Q}$ gives $\tau(p)(\tau(c_0) + \tau(c_1)\tau(s)) \equiv \tau(M) \pmod{Q}$. The secret key and restoration factor become $\tau(s)$ and $\tau(p)$, respectively. To restore with p, choose a public $q_\tau \in R_Q$ satisfying $pq_\tau = \tau(p)$, when a solution exists, and return

$$\mathsf{ct}' = \mathsf{KS}_{\tau(s)\to s}([q_\tau \tau(c_0)]_Q, [q_\tau \tau(c_1)]_Q).$$

If key switching adds phase error $\kappa_\tau \in R$, its output satisfies $c_0' + c_1' s \equiv q_\tau(\tau(c_0) + \tau(c_1)\tau(s)) + \kappa_\tau \pmod{Q}$. Restoration multiplies this relation by p; using $pq_\tau = \tau(p)$ gives $M' = \tau(M) + p\kappa_\tau$. One may take $q_\tau = \tau(p)p^{-1}$ when p is invertible modulo Q, or $q_\tau = 1$ when $\tau(p) = p$, even without invertibility. The output retains modulus Q and scale Δ and represents the permutation and possible conjugation of the logical slots induced by τ.

For a linear transformation $\mathcal{L}$ on the logical slots, write the stored message vector as $\mathbf{z} = U^{-1}\mathbf{x}$, where $U = \mathrm{diag}(\sigma_0(p), \ldots, \sigma_{r-1}(p))$. The desired output $\mathcal{L}\mathbf{x}$ must be stored as $U^{-1}\mathcal{L}\mathbf{x} = U^{-1}\mathcal{L}U\mathbf{z}$. If $\mathcal{L}U = U\mathcal{L}$, this equals $\mathcal{L}\mathbf{z}$, so the same map $\mathcal{L}$ can be applied directly to the stored values. This holds for sums and averages within groups whose values $\sigma_j(p)$ are equal, since their common factor $1/\sigma_j(p)$ can be taken outside each sum. Equal absolute values do not suffice, since signs and complex phases also affect the sum. The absolute values $|\sigma_j(p)|$ control precision allocation, while the full values $\sigma_j(p)$ determine the corrections that preserve the logical computation.

If every rotation used to implement $\mathcal{L}$ fixes p, then these rotations use $q_\tau = 1$ and require no public correction.

The following lemma gives the exact polynomial after rescaling. It does not require p to be invertible or the input phase $c_0 + c_1 s$ to lie in the centered interval modulo Q. Its proof is given in Appendix A.

Lemma 7.1 (Preservation under rescaling)**.** *Let $Q = DQ'$ with odd $D > 1$, and let $\mathsf{ct} = (c_0, c_1)$ be a ciphertext at modulus Q and scale $\Delta > 0$. Suppose that $M \in R$ satisfies $p(c_0 + c_1 s) \equiv M \pmod{Q}$ and $\sigma(M) = \Delta(\mathbf{x} + \mathbf{e})$. Let ρ_0, ρ_1 and $\mathsf{ct}' = (c_0', c_1')$ be as in RS_D. Then*

$$M' = \frac{M - p(\rho_0 + \rho_1 s)}{D} \tag{17}$$

belongs to R and satisfies $p(c_0' + c_1' s) \equiv M' \pmod{Q'}$. With $\Delta' = \Delta/D$, its embeddings satisfy $\sigma_j(M')/\Delta' = x_j + e_j'$ for every slot j, where

$$e_j' = e_j - \frac{\sigma_j(p)\sigma_j(\rho_0 + \rho_1 s)}{D\Delta'}.$$

For multiplication followed by rescaling, the lemma gives

$$M' = \frac{M_a M_b + p\kappa - p(\rho_0 + \rho_1 s)}{D}, \qquad \Delta' = \frac{\Delta_a \Delta_b}{D}.$$

Both new error terms carry the factor p upon restoration. Again, p is a polynomial in R, so under σ this factor becomes $\sigma_j(p)$ in slot j. Bounds for these terms and for the propagation of existing errors are deferred to Section 7.2.

7.2 Error Analysis and Security

We bound the difference between the recovered value and the intended result in each slot, and give conditions that guarantee the required precision. We use the node and slot notation of Section 5 and the error bounds from Section 6.1. For $\mathsf{ct}_v = (c_{v,0}, c_{v,1})$, the restored polynomial $M_v \in R$ satisfies $M_v \equiv p(c_{v,0} + c_{v,1}s) \pmod{Q_v}$. The restored error in slot j is $e_{v,j} = \sigma_j(M_v)/\Delta_v - x_{v,j} = \sigma_j(p)\eta_{v,j}$. The bounds $A_{v,j}, B_{v,j}$ are used only for analysis; intermediate ciphertexts need not be decrypted.

For a fresh ciphertext encrypting $\mathbf{x}_v$, let $e_{\mathrm{enc},v}$ and $e_{\mathrm{dec},v}$ denote its encoding and decryption errors. At an evaluation step v, let κ_v denote the phase error introduced by relinearization or key switching, and let $\rho_{v,0}, \rho_{v,1}$ be the residues used in rescaling.[1] The primitive bounds from Sections 5.1 and 6.1 satisfy, for every slot j,

$$\begin{aligned} |\sigma_j(e_{\mathrm{enc},v})| &\le B_{\mathrm{enc},v}, & |\sigma_j(e_{\mathrm{dec},v})| &\le B_{\mathrm{dec},v}, \\ |\sigma_j(\kappa_v)| &\le K_v, & |\sigma_j(\rho_{v,0} + \rho_{v,1}s)| &\le R_v. \end{aligned} \tag{18}$$

These bounds are taken before restoration and division by the message scale. For exact coefficient rounding, one may take $B_{\mathrm{enc},v} = \Gamma_N/2$. In particular, the rescaling phase error is $-(\rho_{v,0}+\rho_{v,1}s)/D_v$, whose embeddings are bounded in absolute value by R_v/D_v. For the key-switching implementation in Appendix C, K_v can be chosen using (22). We assume that the bounds hold simultaneously for the actual operation inputs, including those after public corrections. For a fresh ciphertext, the triangle inequality gives the initial bound

$$|\eta_{v,j}| = \frac{|\sigma_j(e_{\mathrm{enc},v} + e_{\mathrm{dec},v})|}{\Delta_v} \le \frac{B_{\mathrm{enc},v} + B_{\mathrm{dec},v}}{\Delta_v} =: B_{v,j}.$$

We analyze multiplication and rescaling together:

$$\mathsf{ct}_v = \mathsf{RS}_{D_v}(\mathsf{Relin}_{\mathrm{evk}}(p(\mathsf{ct}_a \otimes \mathsf{ct}_b))), \qquad \Delta_v = \frac{\Delta_a \Delta_b}{D_v}. \tag{19}$$

Here $\otimes$ forms the usual product with three components, and p multiplies each component before relinearization. The inputs share a modulus and secret key, and $D_v > 1$ is an odd divisor of that modulus, as required by RS_{D_v}.

Lemma 7.2 (Error after multiplication and rescaling)**.** *Consider the step in (19). Assume the input bounds $|x_{a,j}| \le A_{a,j}$, $|x_{b,j}| \le A_{b,j}$, $|\eta_{a,j}| \le B_{a,j}$, and $|\eta_{b,j}| \le B_{b,j}$, together with the primitive bounds in (18). Then $x_{v,j} = x_{a,j}x_{b,j}$, and for every slot j,*

$$|e_{v,j}| \le |\sigma_j(p)| \left(A_{a,j}B_{b,j} + A_{b,j}B_{a,j} + |\sigma_j(p)|B_{a,j}B_{b,j} + \frac{K_v + R_v}{D_v\Delta_v} \right). \tag{20}$$

The expression in parentheses is a valid choice of $B_{v,j}$.

The proof is given in Appendix A. The first three terms propagate existing errors, while the last accounts for relinearization and rescaling in this step. The outer factor $|\sigma_j(p)|$ comes from restoration. Starting with the input bounds, we propagate $B_{v,j}$ in circuit order to obtain $B_{\mathrm{out},j}$. For addition at a common scale, take $B_{v,j} = B_{a,j} + B_{b,j}$. Appendix B gives the remaining rules, including approximation of public constants and movement between slots.

[1] For input components c_i modulo D_vQ_v, we have $\rho_{v,i} = [c_i]_{D_v}$ and $c_{v,i} = [(c_i - \rho_{v,i})/D_v]_{Q_v}$ for $i \in \{0, 1\}$.

Evaluation and recovery. The propagated bounds depend on the unit and the scales, so the scale reduction in Proposition 5.1, which assumes a fixed common error bound, cannot be applied independently at every node. For each candidate unit and parameter set, the propagated bounds must satisfy

$$|\sigma_j(u)|B^{\mathrm{u}}_{\mathrm{out},j} \le 2^{-b_j} \qquad \text{for each required output slot } j.$$

The chosen parameters must also satisfy the operation requirements in Section 7.1 and the implementation conditions in Section 7.3. The error bounds must hold for the actual parameters and operation inputs. In particular, the key-switching bound in Appendix C must be recomputed whenever the ciphertext modulus Q, the auxiliary key-switching modulus P, or the decomposition changes. If the bounds $|\sigma_j(u)|B^{\mathrm{u}}_{\mathrm{out},j} \le 2^{-b_j}$ hold for all slots and

$$\frac{Q_{\mathrm{out}}}{\Delta_{\mathrm{out}}} > \frac{2}{r}\sum_{j=0}^{r-1}(A_{\mathrm{out},j} + 2^{-b_j}), \tag{21}$$

then the recovery condition in (7) follows from (4). Ordinary CKKS and our method can use this bound, but their final ratios need not coincide. If dummy slots have no precision targets, use the full recovery condition in (7), including their errors.

Security. The following proposition establishes security.

Proposition 7.1 (Security Reduction)**.** *Fix the public parameters and p before CKKS key generation. If a PPT adversary $\mathcal{A}$ breaks the IND-CPA security of our construction, then a PPT adversary $\mathcal{B}_p$ breaks CKKS:*

$$\mathrm{Adv}^{\text{IND-CPA}}_{\text{ours},p,\text{evk}}(\mathcal{A}) = \mathrm{Adv}^{\text{IND-CPA}}_{\text{CKKS, evk}}(\mathcal{B}_p).$$

Proof. The reduction $\mathcal{B}_p$ interacts with the CKKS challenger $\mathcal{C}$ as follows.

1. $\mathcal{B}_p$ receives (pk, evk) from $\mathcal{C}$ and gives (pk, evk, p) to $\mathcal{A}$.
2. $\mathcal{A}$ sends $(\mathbf{x}_0, \mathbf{x}_1)$ to $\mathcal{B}_p$, which computes $\mu_t = \left[\lfloor \Delta\, \sigma^{-1}((x_{t,j}/\sigma_j(p))_j)\rceil\right]_Q, t \in \{0,1\}$, and submits (μ_0, μ_1) to $\mathcal{C}$.
3. $\mathcal{C}$ returns $\mathsf{ct} = \mathsf{Enc}_{pk}(\mu_b)$ for its hidden bit $b \in \{0,1\}$. $\mathcal{B}_p$ forwards ct to $\mathcal{A}$.
4. $\mathcal{A}$ outputs a guess $b' \in \{0,1\}$, and $\mathcal{B}_p$ returns the same b' to $\mathcal{C}$.

For each b, $\mathcal{A}$'s view has the same distribution as in our scheme. Hence the advantages are equal. □

7.3 Computation and Storage Costs

We compare costs using each method's own parameters that meet the same precision targets for the required output slots and the same security requirement. Smaller moduli may reduce the number of primes used to represent ciphertexts, while public corrections add modular products.

RNS Representation and Parameters. A residue number system (RNS) stores a polynomial separately modulo each prime factor of its modulus [CHK+19]. Write $Q = \prod_{i=0}^{L_Q-1} q_i$ for the ciphertext modulus and $P = \prod_{i=0}^{L_P-1} p_i$ for the auxiliary key-switching modulus; L_Q and L_P are their respective prime counts. The primes are pairwise distinct, fit in one machine word, and satisfy $q_i \equiv p_j \equiv 1 \pmod{2N}$ to support the number theoretic transform (NTT).

We use an RNS decomposition with blocks of $L_P \geq 1$ primes of Q, except possibly the last block, and no further binary decomposition [Tun26]. There are $\beta := \lceil L_Q/L_P \rceil$ blocks. Each block uses an evaluation key pair $(b_k, a_k) \in R_{QP}^2$, so the key contains 2β polynomials. Key switching computes modulo QP and returns a ciphertext modulo Q at the same scale. Appendix C gives the procedure, error bounds, and operation counts.

A polynomial modulo Q occupies NL_Q residue coefficients. In NTT form, one polynomial multiplication uses NL_Q pointwise modular products. We count one NTT or inverse NTT of length N modulo one prime as one transform. An RNS conversion from modulus A to B computes $[a]_B$ for each coefficient, where a is its chosen integer representative modulo A. Modular products, transforms, and RNS conversions are counted separately; the first count excludes work inside the other two.

Computational Cost. Table 1 lists only operations with different counting formulas.

Table 1: Differences in modular product counts between ordinary CKKS and our method. The moduli Q, P and their prime counts L_Q, L_P may differ between methods.

Operation	Ordinary CKKS	Our Method
Mult	$N^{\mathrm{o}}(6L_Q^{\mathrm{o}} + 2L_{Q'}^{\mathrm{o}} + 2\beta^{\mathrm{o}}(L_Q^{\mathrm{o}} + L_P^{\mathrm{o}}))$	$N^{\mathrm{u}}(9L_Q^{\mathrm{u}} + 2L_{Q'}^{\mathrm{u}} + 2\beta^{\mathrm{u}}(L_Q^{\mathrm{u}} + L_P^{\mathrm{u}}))$
Auto ($q_\tau \neq 1$)	$2N^{\mathrm{o}}(L_Q^{\mathrm{o}} + \beta^{\mathrm{o}}(L_Q^{\mathrm{o}} + L_P^{\mathrm{o}}))$	$2N^{\mathrm{u}}(2L_Q^{\mathrm{u}} + \beta^{\mathrm{u}}(L_Q^{\mathrm{u}} + L_P^{\mathrm{u}}))$
Restore	0	$2N^{\mathrm{u}}L_{Q_{\mathrm{out}}}^{\mathrm{u}}$

All operands and outputs use NTT form. Ciphertext multiplication includes relinearization and one rescaling from Q to $Q' = Q/D$, with each method's own moduli and decomposition. The counts for our method include the public corrections; conditions on q_τ refer to our method.

In NTT form, multiplying one component by a public ring element uses N^{u} modular products per prime. Multiplication produces three components, each multiplied by u before relinearization, giving $3N^{\mathrm{u}}L_Q^{\mathrm{u}}$ additional modular products. An automorphism multiplies its two components by q_τ before key switching, adding $2N^{\mathrm{u}}L_Q^{\mathrm{u}}$ modular products when $q_\tau \neq 1$. The factors u and q_τ introduce no additional encoding scale, so these corrections preserve the scale and need no extra rescaling.

The remaining operations have unchanged costs at equal parameters. For $h \in \{\mathrm{o}, \mathrm{u}\}$, addition uses $2N^h L_Q^h$ modular additions, and MultPlain uses $2N^h L_Q^h$ modular products; neither requires transforms or RNS conversions. Auto with $q_\tau = 1$ uses $2N^h(L_Q^h + \beta^h(L_Q^h + L_P^h))$ modular products. All automorphisms also permute $2N^h L_Q^h$ entries.

Public corrections add no transforms or RNS conversions. Mult uses $(\beta^h + 2)(L_Q^h + L_P^h) + 2L_Q^h$ transforms and $\beta^h + 4$ RNS conversions; Auto uses $(\beta^h + 2)(L_Q^h + L_P^h)$ transforms and $\beta^h + 2$ conversions. MultPlain and Restore use neither. These formulas are evaluated with each method's own parameters, so the resulting counts may differ. Squaring reuses the product c_0c_1 and requires $N^h L_Q^h$ fewer modular products than Mult, with all other counts unchanged.

For the L successive squarings in Section 5.1, sum the adjusted modular product counts at each step's modulus and add final restoration. Within our total, the public corrections and restoration

account for

$$3N^{\mathrm{u}} \sum_{t=1}^{L} L^{\mathrm{u}}_{Q_{t-1}} + 2N^{\mathrm{u}} L^{\mathrm{u}}_{Q_L}.$$

Storage Cost For storage, we count uncompressed residue coefficients in 64-bit words, with eight bytes per coefficient. A ciphertext with two components at modulus Q and one switching key with β polynomial pairs at modulus QP require, respectively,

$$S_{\mathrm{ct}} = 16NL_Q \text{ bytes}, \qquad S_{\mathrm{evk}} = 16N\beta(L_Q + L_P) \text{ bytes}.$$

For a public key stored as two polynomials modulo QP, the corresponding size is $16N(L_Q + L_P)$ bytes.

At fixed N, a smaller Q may allow fewer RNS primes and therefore smaller ciphertexts. If L_Q remains unchanged, their storage in machine words is unchanged. Changing L_P affects both β and the width $L_Q + L_P$ of each key component. Key size can therefore change discontinuously, and the parameters with the smallest keys need not give the shortest running time.

Our construction of u also requires extra computation and storage. We count constructing u, computing $\sigma_j(u)$, and preparing public NTT representations separately from homomorphic evaluation. Each stored NTT polynomial modulo Q uses $8NL_Q$ bytes. We also count the storage for the description of u and the embeddings $(\sigma_j(u))_j$ used by the encoder.

8 Implementation

We implement our method in Go with Lattigo v6.2.0 [Tun26]. Methods run one at a time on the same shared Intel Xeon Gold 6240R node, each using one thread. We change their order between repetitions. Online time includes every batch, public corrections, and conversion of the output to ordinary CKKS ciphertexts. It excludes parameter search, unit preparation, key generation, encoding, encryption, decryption, and accuracy checks. We target 128-bit classical security using the Lattice Estimator [APS15] with the full modulus QP, including the auxiliary key-switching modulus P.[2] Our code is publicly available.

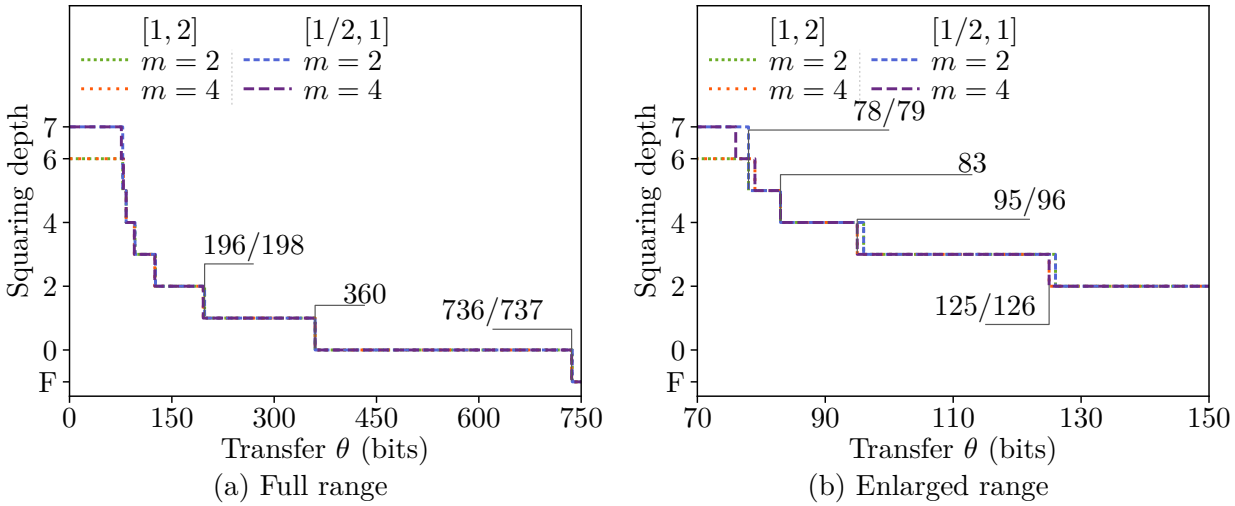
In the first four experiments, our method divides the slots into $m \in \{2, 4\}$ equal classes: one holds useful inputs and the others contain zeros. In the ideal allocation, each dummy class gives up θ bits of precision and the useful class gains $(m-1)\theta$ bits. Reported precision is $-\log_2 \max_j |\widehat{x}_j - x_j|$, where the maximum is over useful outputs. We compute reference values with exact rational arithmetic and decode outputs at high precision.

8.1 Precision Transfer and Recoverable Depth

We first count how many successive ciphertext squarings we can perform before decryption fails, varying θ while keeping the other parameters fixed. We fix $N = 2^{15}$, $\log_2 Q = 744$, $\log_2 P = 112$, and $\Delta_0 = 2^{92}$. Each squaring removes two primes from Q, and the chain supports at most seven squarings. The 4096 useful inputs are $x_i = 1 + i/4095$, or their reciprocals. The average precision target remains 64 bits. After each step, we check that every coefficient of M_v, before reduction modulo Q_v, has absolute value below $Q_v/2$. This determines correct decryption; output precision

[2] We use estimator commit `601905601`, full RLWE dimension N, uniform ternary secrets unless stated otherwise, and an unbounded number of samples. For $q_{\mathrm{est}} = 2^{\lceil \log_2(QP) \rceil}$, we take the minimum over primal-uSVP [ADPS16], primal-BDD [LN13], and dual-hybrid estimates under the `RC.MATZOV` cost model [MAT22]. Sampled errors have standard deviation $\sigma_{\mathrm{err}} = 3.2$ and truncation bound $6\sigma_{\mathrm{err}} = 19.2$; the estimator uses the untruncated Gaussian model following [BCC+25].

is recorded separately. Figure 1 reports median depths from three runs, using five near selected changes in depth. Within each repetition, we use the same keys and encryption noise across allocations.[3]



(a) Full range (b) Enlarged range

Figure 1: Recoverable squaring depth versus transfer θ.

For small transfers, the number of successful squarings stays the same as at $\theta = 0$; larger transfers reduce this number. Two and four classes support similar depths at the same θ, although their ideal useful-slot precision gains are θ and 3θ, respectively. Each dummy slot has the same ideal error multiplier 2^{θ} in both layouts. The squared-error term $|\sigma_j(p)|^2 B_{a,j}^2$ in (20) thus has the same ideal factor $2^{2\theta}$, consistent with the similar depths.

Although dummy inputs are zero, squaring also squares their existing errors (Equation (20)). As these errors grow and rescaling reduces the modulus, coefficients of M_v can exceed $Q_v/2$ in absolute value, causing decryption to fail (Equation (7)). Inputs in $[1, 2]$ also grow under squaring, leaving less room for errors, while their reciprocals remain bounded by one. The two input ranges give similar depths once errors in dummy slots become the main cause of failure. This suggests that the growth of useful values is no longer the main factor limiting depth. F marks failure to decrypt even before the first squaring.

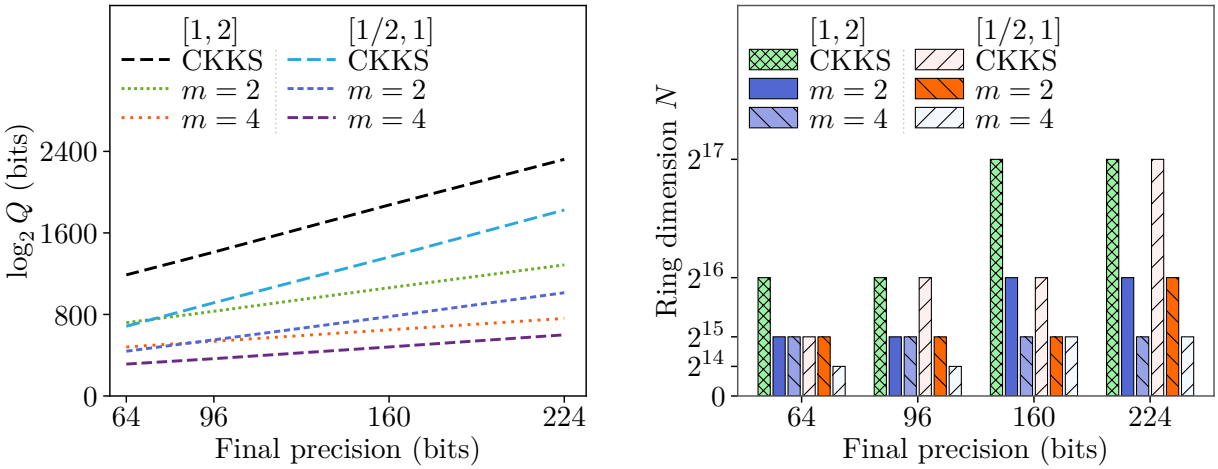
8.2 Parameter and Time Savings

We next compare parameter requirements and online time for six squarings of 256 inputs in $[1, 2]$ or their reciprocals, with precision targets from 64 to 224 bits. All methods search within the same finite parameter ranges. We keep a candidate only if our error bounds guarantee the required precision and correct decryption, and its security estimate meets 128 bits. For Figure 2, we choose the smallest N that passes these checks, then the smallest Q at that N.

Precision allocation lowers Q and often permits a smaller N, with greater savings for four classes. Reducing errors in useful slots lets us use a smaller scale while meeting the same precision target. In these experiments, this reduces the modulus consumed by rescaling and hence the initial Q (Equation (10)). A smaller Q can also allow a smaller N at the same security target. Since the candidate values of N are powers of two, a decrease in Q does not always change the selected N.

At the 224-bit target, the four-class scheme reduces $\log_2 Q$ from 2322 to 761 and N from 2^{17} to 2^{15} (Table 2). Median online time falls from 9.269 to 0.645 seconds, a 14.36× speedup including

[3] We scan $\theta = 0, 10, \ldots, 750$ and use one-bit steps near changes in depth.



(a) Initial modulus (b) Ring dimension

Figure 2: Selected Q and N for six squarings of 256 inputs at different precision targets.

Table 2: Six squarings of 256 inputs in $[1, 2]$, with a 224-bit target.

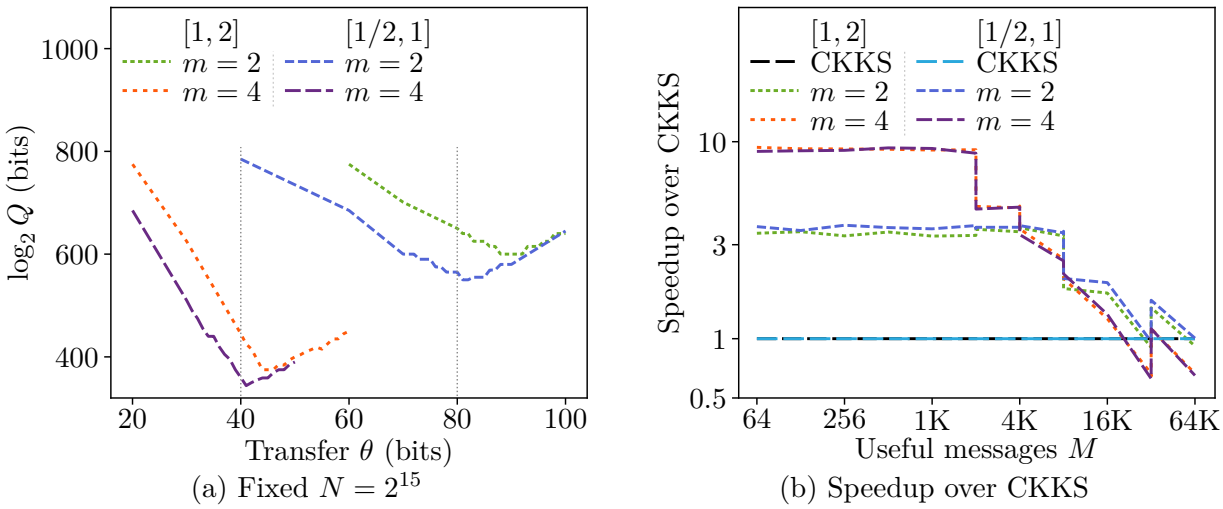
Method	N	$\log_2 Q$	Security (bits)	Precision (bits)	Time (s)	Speedup
CKKS	2^{17}	2322	173.64	237.66	9.269	1.00×
Two classes	2^{16}	1286	139.86	238.45	2.001	4.63×
Four classes	2^{15}	761	133.59	235.20	0.645	14.36×

public corrections and output conversion. Times are medians of five independent runs; precision is the lowest value observed across those runs. As analyzed in Section 7.3, smaller N means fewer polynomial coefficients to process, and fewer RNS primes mean fewer modular operations.

8.3 Transfer Amount and Message Count

For four squarings at a 160-bit target, we vary the transfer amount and useful message count. For each θ, we search for parameters that pass the preceding precision, decryption, and security checks. Figure 3(a) shows the selected moduli at $N = 2^{15}$ and 256 useful messages. Increasing θ first allows smaller scales and a smaller Q. Beyond a certain point, errors in dummy slots become so large that Q must increase to ensure correct decryption. For the same ideal useful-slot gain, four classes require one-third the θ used by two classes, amplifying the error in each dummy slot less.

In Figure 3(b), we increase the number of useful messages. More dummy slots leave fewer slots for useful messages in each ciphertext, so the same workload may require more ciphertext batches. Before timing, we choose the parameters with the lowest estimated work for all batches among those passing the checks above. Speedups compare median total times from five runs, with all batches executed one after another. For inputs in $[1, 2]$, the four-class scheme achieves about 9.2× at 256 messages, but only 0.67× at 65536. The extra batches then make it slower than ordinary CKKS, despite the lower cost per batch. Sudden changes occur when the batch count or selected parameters change. However, with sufficient computing resources, running these batches in parallel may make our method faster than ordinary CKKS.

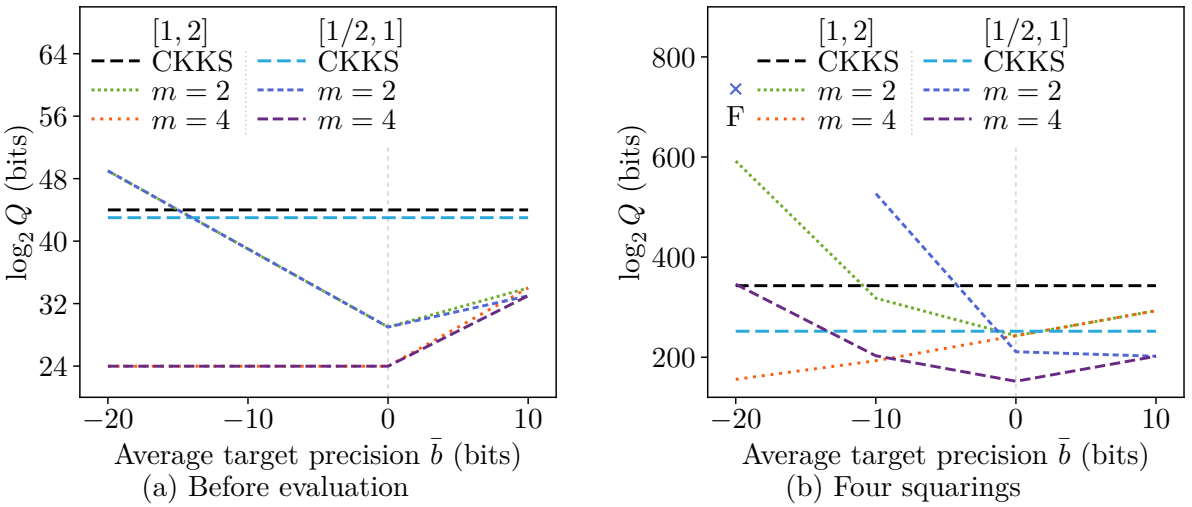


(a) Fixed $N = 2^{15}$ (b) Speedup over CKKS

Figure 3: Four squarings at a 160-bit target: modulus versus transfer θ (a); speedup over CKKS versus useful message count M (b).

8.4 Zero and Negative Average Precision Targets

We fix the useful-slot target at 20 bits and test average targets $\bar{b} \in \{10, 0, -10, -20\}$, with $N = 2^{15}$ and 256 inputs in $[1, 2]$ or their reciprocals. The average includes targets for dummy slots, used only to choose the unit and scale. We do not require dummy outputs to meet them. We search for Q under fixed scale and rescaling rules. Each reported candidate passes five independent runs: the entire polynomial decrypts correctly and all useful outputs reach 20 bits. The general sufficient error bounds do not prove correctness for these parameters; the results here are verified by actual runs.



(a) Before evaluation (b) Four squarings

Figure 4: Candidate initial moduli at a 20-bit useful-slot target and different average targets $\bar{b}$.

The successful runs show that useful outputs can reach 20 bits even when $\bar{b}$ is negative. A more negative $\bar{b}$, however, does not always give a smaller Q. For inputs in $[1, 2]$ after four squarings, the four-class scheme at $\bar{b} = -20$ uses about 156 bits of modulus, versus 592 for two classes; ordinary CKKS uses about 343 bits. This is consistent with (11): larger errors in dummy slots can require a larger final ratio Q_L/Δ_L for recovery, offsetting the savings from smaller scales. At a given $\bar{b}$, four

classes assign smaller target error multipliers to dummy slots than two classes. This can reduce the Q needed to decrypt the entire polynomial correctly. The flat 24-bit segment in Figure 4(a) is the smallest modulus size allowed in the search. F in panel (b) marks decryption failure even at the largest tested modulus, with $\log_2 Q \approx 736$.

8.5 Comparison with Tuple CKKS

Finally, we compare our two-class scheme with a Lattigo-based reference implementation of $t = 2$ Tuple CKKS for nonzero messages with different precision requirements.[4] Here $t = 2$ means that Tuple CKKS represents each message vector with two ciphertexts. Both methods use $N = 2^{15}$, sparse ternary secrets with exactly 128 randomly signed nonzero coefficients, and eight successive squarings of inputs in $[255/256, 1)$. Both methods must return 8192 outputs with at least 80 bits of precision. The remaining outputs have targets of 1 bit for our scheme and 32 bits for Tuple. We set $\Delta_0 = 2^{70}$ and $\theta = 24 \log_2(1 + \sqrt{2})$. For useful slots, $\Delta_0/|\sigma_j(u)| = 2^{70+\theta}$, close to Tuple's scale 2^{100}.

We repeat the comparison ten times with the same inputs for both methods and new keys each time. We alternate which method runs first. Table 3 reports mean online time with sample standard deviation and the lowest precision over the 8192 outputs and all runs. Timings include Tuple's conversion to ordinary CKKS. Storage includes only the stored polynomial coefficients.

Table 3: Eight squarings with 8192 high-precision outputs at an 80-bit target. Logarithms, security, and precision are in bits. Output storage is measured before conversion to ordinary CKKS.

Method	Parameters				Performance		Storage (MiB)	
	$\log_2 Q$	$\log_2(QP)$	$\log_2 \Delta_0$	Security	Precision	Time (s)	Keys	Output
Two classes	680	720	70	132.54	80.61	$\mathbf{1.099} \pm 0.037$	**112.50**	**1.00**
Tuple CKKS	622	682	100	138.04	80.12	1.548 ± 0.023	127.00	2.00

Both methods meet the 80-bit target. Our scheme reduces mean online time by 29% and key storage by 11.4%, despite its larger modulus. Our method stores the packed messages in one ciphertext. Tuple's two ciphertexts require extra products between them and decomposition steps. Our evaluation uses eight relinearizations and one relinearization key, compared with sixteen and two for Tuple. Before conversion to ordinary CKKS, our output takes half as much storage. After conversion, both occupy 1 MiB. This comparison shows that precision allocation can also reduce computation when every slot carries a message, provided lower precision is acceptable for some outputs.

[4] We use the parameters of [CCKS23, Section 6.3, Table 3], increasing the two base primes from 50 to 51 bits, giving $\log_2(QP) \approx 682$. Tuple's Q includes its 40-bit decomposition prime.

Appendix

A Deferred Proofs

Proof of Equation (4)

Proof. Write $f = \sum_{h=0}^{N-1} f_h X^h$ with $f_h \in \mathbb{R}$. Put $\xi = e^{\pi i/N}$ and first suppose $\max_h |f_h| \le 1$. Since $|z| = \max_\phi \mathrm{Re}(e^{-i\phi} z)$,

$$|f(\xi)| \le \max_{\phi} \sum_{h=0}^{N-1} |\cos(h\pi/N - \phi)| \,.$$

When none of these cosines vanish, choosing $\epsilon_h = \operatorname{sgn} \cos(h\pi/N - \phi)$ selects the N consecutive vertices $\epsilon_h \xi^h$ of the regular $2N$-gon in the open semicircle centered at angle ϕ. Their sum has modulus $2/|1-\xi| = \Gamma_N$, so its projection in direction ϕ, the sum above, is at most Γ_N. The remaining directions follow by continuity. Equality holds for $f = 1 + X + \cdots + X^{N-1}$ at ξ. For any odd a, the map $h \mapsto ah \bmod N$ permutes the indices, and $\xi^{ah} = (-1)^{\lfloor ah/N \rfloor} \xi^{ah \bmod N}$. Thus evaluation at ξ^a differs only by a signed permutation of the coefficients and has the same optimal constant. Homogeneity gives the forward bound for all f.

For the reverse bound, let $\omega_t := e^{(2t+1)\pi i/N}$, $0 \le t < N$, be the roots of $X^N + 1$. For $0 \le h, k < N$, geometric summation gives $\frac{1}{N} \sum_{t=0}^{N-1} \omega_t^{h-k} = \delta_{hk}$. Fourier inversion and conjugate pairing, using $f_h \in \mathbb{R}$ and $N = 2r$, give

$$f_k = \frac{1}{N} \sum_{t=0}^{N-1} f(\omega_t) \omega_t^{-k} = \frac{1}{r} \sum_{j=0}^{r-1} \mathrm{Re}(\sigma_j(f) \sigma_j(X)^{-k}).$$

Thus $|f_k| \le \frac{1}{r} \sum_{j=0}^{r-1} |\sigma_j(f)| \le \|f\|_{\sigma,\infty}$ for every k. Taking the maximum over k completes (4). The multiplication bound follows from $\sigma_j(fg) = \sigma_j(f)\sigma_j(g)$ by taking absolute values and the maximum over j. □

Proof of Lemma 3.1

Proof. Equation (5) and the hypothesis give

$$\|p(\mu + e_{\mathrm{dec}})\|_{\mathrm{coef},\infty} < \frac{Q}{2}.$$

Since $p(c_0 + c_1 s) \equiv p(\mu + e_{\mathrm{dec}}) \pmod{Q}$, centered decryption returns $p(\mu + e_{\mathrm{dec}})$. Applying σ_j/Δ to (5) gives the claimed decoded values. □

Proof of Proposition 3.1

Proof. Applying (4) to $f = p(e_{\mathrm{enc}} + e_{\mathrm{dec}})$ and using the assumed embedding bounds yields

$$\|p(e_{\mathrm{enc}} + e_{\mathrm{dec}})\|_{\mathrm{coef},\infty} \le \frac{1}{r} \sum_{j=0}^{r-1} |\sigma_j(p)| \, |\sigma_j(e_{\mathrm{enc}} + e_{\mathrm{dec}})| \le \frac{\Delta}{r} \sum_{j=0}^{r-1} |\sigma_j(p)| B_j \le \frac{\Delta}{r} \sum_{j=0}^{r-1} \varepsilon_j.$$

Under (6), Lemma 3.1 gives recovery with $|\widehat{x}_j - x_j| \le |\sigma_j(p)| B_j \le \varepsilon_j$. □

Proof of Modulus Invariance under Equivalent Encoding

Proposition A.1 (Modulus length under equivalent encoding)**.** *Fix the ring, p, and a circuit evaluated as in Section 7.1, with Δ_t the scale at node t. With identical rounding and exact scale bookkeeping, the rewriting $(\Delta_t, \sigma_j(p)) \mapsto (\Delta_t/\lambda, \sigma_j(p)/\lambda)$ [5] for a common $\lambda > 0$ leaves the feasible modulus chains and their total lengths unchanged.*

Proof. For any feasible original execution, use the same input modulus, keys, and randomness. Tildes denote the rewritten form, with $\widetilde{\Delta}_t = \Delta_t/\lambda$. Input and additive-plaintext encodings give $\widetilde{\mu} = \mu$, since $(\Delta/\lambda)(\lambda x_j/\sigma_j(p)) = \Delta x_j/\sigma_j(p)$ and rounding is identical. At a multiplication node c with inputs a, b, the correction p has plaintext scale λ, so the rescaling divisor is

$$\widetilde{D} = \frac{\lambda\widetilde{\Delta}_a\widetilde{\Delta}_b}{\widetilde{\Delta}_c} = \frac{\lambda(\Delta_a/\lambda)(\Delta_b/\lambda)}{\Delta_c/\lambda} = \frac{\Delta_a\Delta_b}{\Delta_c} = D.$$

For the remaining rules, retain the multiplicative plaintext scale Δ_{pt} and the same public corrections. Addition preserves scale alignment because $\widetilde{\Delta}_a = \widetilde{\Delta}_b \iff \Delta_a = \Delta_b$. Plaintext multiplication and rescaling preserve the correspondence: $\widetilde{\Delta}_t\Delta_{\mathrm{pt}} = (\Delta_t\Delta_{\mathrm{pt}})/\lambda$ and $\widetilde{\Delta}_t/D = (\Delta_t/D)/\lambda$. Automorphisms and linear maps use $\widetilde{q}_\tau = q_\tau$ and $(U/\lambda)^{-1}\mathcal{L}(U/\lambda) = U^{-1}\mathcal{L}U$. Induction on the integer operations gives $(\widetilde{\mathsf{ct}}_t, \widetilde{Q}_t, \lambda\widetilde{\Delta}_t) = (\mathsf{ct}_t, Q_t, \Delta_t)$.

Recovery then gives $\widehat{\widetilde{x}}_j = \sigma_j(\mathsf{Dec}_s(p\mathsf{ct}_{\mathrm{out}}))/(\lambda\widetilde{\Delta}_{\mathrm{out}}) = \widehat{x}_j$. This reversible correspondence preserves every modulus Q_t, hence the feasible modulus chains and their total bit lengths. □

Proof of Proposition 6.1

Proof. Using the common bound B_{out}, the recovery condition in (7) holds if $t + t^{-1} < C$. For $C > 2$ and $t = 2^\theta \geq 1$,

$$t + t^{-1} < C \iff t^2 - Ct + 1 < 0 \iff 1 \leq t < \frac{C + \sqrt{C^2 - 4}}{2}.$$

Taking logarithms gives the stated bound. □

Proof of Lemma 7.1

Proof. Choose integer representatives $c_0, c_1 \in R$. The assumed congruence gives $I \in R$ such that

$$p(c_0 + c_1 s) = M + DQ'I.$$

Since $\rho_i \equiv c_i \pmod{D}$, reduction modulo D gives $M \equiv p(\rho_0 + \rho_1 s) \pmod{D}$. Thus the numerator in (17) is divisible by D in R, so $M' \in R$. Subtracting $p(\rho_0 + \rho_1 s)$ and dividing by D gives

$$p\left(\frac{c_0 - \rho_0}{D} + \frac{c_1 - \rho_1}{D}s\right) = M' + Q'I.$$

Reduction modulo Q' proves the claimed congruence. Applying σ_j to (17) and dividing by $\Delta' = \Delta/D$ gives the stated embedding relation and error formula. For arbitrary $h_i \in R$, replacing c_i by $c_i + Qh_i$ leaves ρ_i unchanged and changes $(c_i - \rho_i)/D$ by $Q'h_i$, so the output is independent of the representatives. □

[5] The identity $\sigma_j(p/\lambda) = \sigma_j(p)/\lambda$ holds for every $\lambda > 0$, even when $p/\lambda \notin R$.

Proof of Lemma 7.2

Proof. Apply Lemma 7.1 to the polynomial $M_a M_b + p\kappa_v$ after relinearization. Expanding the resulting embedding relation and using $e_{a,j} = \sigma_j(p)\eta_{a,j}$, $e_{b,j} = \sigma_j(p)\eta_{b,j}$, and $D_v\Delta_v = \Delta_a\Delta_b$ gives

$$\eta_{v,j} = x_{a,j}\eta_{b,j} + x_{b,j}\eta_{a,j} + \sigma_j(p)\eta_{a,j}\eta_{b,j} + \frac{\sigma_j(\kappa_v - \rho_{v,0} - \rho_{v,1}s)}{D_v\Delta_v}.$$

Taking absolute values and applying the assumed bounds shows that the expression in parentheses in (20) bounds $|\eta_{v,j}|$. Multiplication by $|\sigma_j(p)|$, using $e_{v,j} = \sigma_j(p)\eta_{v,j}$, proves (20). □

B Additional Error Propagation

We use the notation and primitive bounds of Section 7.2. The following rules give bounds at output node v from the bounds at its input nodes. All scales follow the exact updates in Section 7.1.

$\mathsf{AddPlain}(\mathsf{ct}_a, \mathbf{a})$. For addition of a public vector $\mathbf{a}$, write $\mu_+ = \Delta_a\sigma^{-1}((a_j/\sigma_j(p))_j) + e_+$ and let $B_{+,v}$ bound $|\sigma_j(e_+)|$ in every slot. Coefficient rounding permits $B_{+,v} = \Gamma_N/2$. Since $M_v = M_a + p\mu_+$, one may take $B_{v,j} = B_{a,j} + B_{+,v}/\Delta_a$.

$\mathsf{MultPlain}(\mathsf{ct}_a, \mathbf{a}; \Delta_{\mathrm{pt}})$. For multiplication by a public vector $\mathbf{a}$ encoded at scale Δ_{pt}, define its encoding discrepancy $\delta_j = \sigma_j(\mu_\times)/\Delta_{\mathrm{pt}} - a_j$ and choose $\beta_{v,j} \geq |\delta_j|$. Coefficient rounding permits $\beta_{v,j} = \Gamma_N/(2\Delta_{\mathrm{pt}})$; a sharper certified bound may be used when available. The identity $M_v = \mu_\times M_a$ gives $\eta_{v,j} = (a_j + \delta_j)\eta_{a,j} + x_{a,j}\delta_j/\sigma_j(p)$, hence

$$B_{v,j} = (|a_j| + \beta_{v,j})B_{a,j} + \frac{A_{a,j}\beta_{v,j}}{|\sigma_j(p)|}.$$

Thus approximation of the public multiplier contributes an error whose restored bound includes $A_{a,j}\beta_{v,j}$.

$\mathsf{Mult}_{\mathrm{evk}}(\mathsf{ct}_a, \mathsf{ct}_b)$. For multiplication without rescaling, the same expansion as in the proof of Lemma 7.2 gives (20) with $D_v = 1$ and $R_v = 0$. Rescaling already included in that lemma is not counted again.

$\mathsf{RS}_{D_v}(\mathsf{ct}_a)$. For rescaling alone, Lemma 7.1 gives $B_{v,j} = B_{a,j} + R_v/(D_v\Delta_v)$.

$\mathsf{ModReduce}_{Q'}(\mathsf{ct}_a)$. Modulus reduction leaves $B_{v,j} = B_{a,j}$, since M_a and the scale remain unchanged; the smaller modulus must still satisfy the range condition at recovery.

$\mathsf{Auto}_\tau(\mathsf{ct}_a)$. For Auto_τ, assume the public correction in Section 7.1 exists. If τ moves input slot k to output slot j, possibly with conjugation, the identity $M_v = \tau(M_a) + p\kappa_v$ gives

$$B_{v,j} = \frac{|\sigma_k(p)|}{|\sigma_j(p)|}B_{a,k} + \frac{K_v}{\Delta_v}.$$

Conjugation preserves absolute values, so the same bound applies in both cases. If $\tau(p) = p$, the ratio is one.

Linear transformations. Linear transformations are bounded by propagating these rules through their actual sequence of operations.

Primitive bounds. The bounds above do not require independence of the errors. For probabilistic primitive bounds, the failure probability of their joint event is at most the sum of their failure

probabilities. For rescaling by D_v, the centered residues satisfy $\|\rho_{v,i}\|_{\mathrm{coef},\infty} \le (D_v-1)/2$ for $i \in \{0,1\}$. Equation (4) therefore gives $|\sigma_j(\rho_{v,i})| \le \Gamma_N(D_v-1)/2$. If $\|s\|_{\sigma,\infty} \le B_s$, then

$$\begin{aligned}|\sigma_j(\rho_{v,0}+\rho_{v,1}s)| &\le |\sigma_j(\rho_{v,0})| + |\sigma_j(\rho_{v,1})|\,|\sigma_j(s)| \\ &\le \frac{\Gamma_N(D_v-1)}{2}(1+B_s) =: R_v.\end{aligned}$$

Thus $R_v/D_v = \Gamma_N(1+B_s)(1-1/D_v)/2 \le \Gamma_N(1+B_s)/2$.

C Details of RNS Key Switching

We give the procedure and bounds for the RNS decomposition of Section 7.3, using exact conversion and centered residues, followed by the counts in Table 1.

Procedure. Let $\mathsf{ct} = (c_0, c_1)$ be a ciphertext under s'. For block k, let Q_k be the product of its primes and take $d_k := [c_1]_{Q_k} \in R$, for $0 \le k < \beta$. Choose integer weights g_k satisfying $g_k \equiv 1 \pmod{Q_k}$ and $g_k \equiv 0 \pmod{Q/Q_k}$. The Chinese remainder theorem gives $c_1 \equiv \sum_{k=0}^{\beta-1} g_k d_k \pmod{Q}$, and hence $c_1 s' \equiv \sum_{k=0}^{\beta-1} d_k(g_k s') \pmod{Q}$. The evaluation key pair $(b_k, a_k) \in R_{QP}^2$ has error $e_k \in R$ and satisfies $b_k + a_k s \equiv P g_k s' + e_k \pmod{QP}$. Choose integer representatives t_0, t_1 for $\sum_k d_k b_k, \sum_k d_k a_k$ modulo QP, respectively. Then $t_0 + t_1 s \equiv P c_1 s' + \sum_k d_k e_k \pmod{QP}$. Put $\rho_i := [t_i]_P$ for $i \in \{0,1\}$. Modulus descent divides by P with rounding and returns $\mathsf{ct}' = (c_0', c_1')$, where $c_0' = [c_0 + (t_0 - \rho_0)/P]_Q$ and $c_1' = [(t_1 - \rho_1)/P]_Q$. This division cancels the factor P in the key messages, preserving the message scale.

Error bounds. The output satisfies

$$c_0' + c_1' s \equiv c_0 + c_1 s' + \kappa \pmod{Q}, \qquad \kappa = \frac{\sum_k d_k e_k - \rho_0 - \rho_1 s}{P}.$$

The numerator is divisible by P, since $\rho_0 + \rho_1 s \equiv t_0 + t_1 s \equiv \sum_k d_k e_k \pmod{P}$. Thus $\kappa \in R$. With $\xi := -(\rho_0 + \rho_1 s)/P \in K$, the triangle inequality gives

$$\|\kappa\|_{\sigma,\infty} \le \frac{1}{P} \sum_{k=0}^{\beta-1} \|d_k\|_{\sigma,\infty} \|e_k\|_{\sigma,\infty} + \|\xi\|_{\sigma,\infty}. \tag{22}$$

Equation (4) gives $\|d_k\|_{\sigma,\infty} \le \Gamma_N(Q_k-1)/2$ and $\|\rho_i\|_{\sigma,\infty} \le \Gamma_N(P-1)/2$. Consequently,

$$\|\kappa\|_{\sigma,\infty} \le \frac{\Gamma_N}{2P} \sum_{k=0}^{\beta-1} (Q_k - 1)\|e_k\|_{\sigma,\infty} + \frac{\Gamma_N(P-1)}{2P}(1 + \|s\|_{\sigma,\infty}).$$

This bound holds for every $c_1 \in R_Q$, including components multiplied by a public ring element. Relinearization has the same error bound when switching the third component from s^2 to s. When Q, P, or the decomposition changes, we recompute the blocks and the bound. An approximate RNS conversion may use different lifts of the digits or residues [CHK+19]; its corresponding bounds replace the centered bounds above in (22).

Operation counts. We count operations with inputs, outputs, and keys in NTT form, reusing the input NTT values within each decomposition block [Tun26]. Let L_{Q_k} be the number of primes in block k, so $\sum_k L_{Q_k} = L_Q$. Decomposition first uses L_Q inverse transforms. For each d_k, one RNS conversion extends its coefficients from Q_k to QP/Q_k, followed by $L_Q + L_P - L_{Q_k}$ forward transforms; the NTT values modulo Q_k are already available. The transform count is therefore

$L_Q + \sum_k (L_Q + L_P - L_{Q_k}) = \beta(L_Q + L_P)$, with β conversions. Multiplication by the β evaluation key pairs uses $2\beta N(L_Q + L_P)$ modular products.

For modulus descent, each of the two components uses L_P inverse transforms, one conversion from P to Q, and L_Q forward transforms. Subtracting the converted residues and multiplying by $P^{-1} \bmod Q$ uses $2NL_Q$ modular products in total. Thus modulus descent uses $2(L_Q + L_P)$ transforms and two conversions. For rescaling by D in one step, write $Q = DQ'$ and let L_D be the number of removed primes, so $L_D + L_{Q'} = L_Q$. The same steps with bases D and Q' use $2L_Q$ transforms, two conversions, and $2NL_{Q'}$ modular products. Each conversion acts on all N coefficients; its cost depends on the source and destination bases.

D Bootstrapping with Precision Allocation and Ring Switching

Ring switching can reduce bootstrapping cost by performing the expensive steps in a smaller ring [GHPS12, MHWW26, LD26]. Security, however, limits the ciphertext modulus available in that ring. Our method and related concurrent methods can reduce the modulus needed by allowing larger errors in unused slots [CL26, Dum26, MLWW26]. Concurrent work has already applied error redistribution to bootstrapping [CL26, Dum26]. Here we combine the ring-switching method of [LD26] with Dumezy's small-batch transforms and modular reduction [Dum26]. We test this combination on selected parameter sets, with limited parameter optimization.

We split a ciphertext into branches in a smaller ring, refresh those containing messages, and merge them back while preserving precision allocation. In our experiments, bootstrapping from degree 2^{17} through a smaller ring of degree 2^{15} takes 13.06 s in parallel and returns approximately 27-bit precision at output level 13. Direct bootstrapping at degree 2^{13} takes 2.49 s in parallel and returns approximately 3-bit precision at output level 2, with $Q_{\text{out}} > Q_{\text{in}}$.

D.1 Adapting the Bootstrapping Steps

Rings and ciphertext representation. Let N be the original ring degree and $n \le N$ the degree used for bootstrapping. Both are powers of two, and $d = N/n$ is the number of branches. Write $R^{(N)} = \mathbb{Z}[X]/(X^N+1)$ and $R^{(n)} = \mathbb{Z}[Y]/(Y^n+1)$. Following [LD26], split a polynomial by grouping coefficients whose indices are equal modulo d:

$$a(X) = \sum_{j=0}^{d-1} X^j a_j(X^d), \qquad \pi_j(a) = a_j, \qquad \iota(a_j) = a_j(X^d).$$

Here $\pi_j : R^{(N)} \to R^{(n)}$ extracts branch j, and $\iota : R^{(n)} \to R^{(N)}$ substitutes X^d for Y. Extend both maps to real-coefficient ideal messages. Let $\mathcal{M}$ be the public set of allowed ideal messages, before encoding and encryption errors. Write s_N and s_n for the two secret keys, and $\mathsf{KS}_{s \to s'}$ for key switching from s to s'.

Choose a real unit $p_n \in (R^{(n)})^\times$ and set $p_N = \iota(p_n)$. This choice makes multiplication by the unit commute with splitting:

$$\pi_j(p_N a) = p_n \pi_j(a), \qquad p_n \pi_j(p_N^{-1} a) = \pi_j(a). \tag{23}$$

For a main-text ciphertext $\mathsf{ct} = (c_0, c_1)$, we work with $\widehat{\mathsf{ct}} = p\mathsf{ct}$, multiplying both components by the current ring's unit. Its phase is the restored polynomial: $\widehat{c}_0 + \widehat{c}_1 s \equiv M \pmod{Q}$. Here M includes the scaled message and error, as in Section 7.1. Multiplication by p or p^{-1} is exact and consumes no modulus level.

Message layout. We first describe the replicated layout used in the $n \geq 2^{14}$ experiments. Each retained branch holds n_{msg} independent complex messages $z_0, \ldots, z_{n_{\mathrm{msg}}-1}$ and uses m precision classes, where $m \mid n_{\mathrm{msg}}$ and $n_{\mathrm{msg}} m \mid n/2$. Number the slots by $\sigma_k(Y) = e^{\pi i 5^k/n}$ for $0 \leq k < n/2$. Following [Dum26, Section 5], repeat the messages across useful slots: slot mt holds $z_{t \bmod n_{\mathrm{msg}}}$ for $0 \leq t < n/(2m)$. The remaining slots, called guards, have intended value zero. Precision classes are the slot indices modulo m. Choose p_n in the degree-$2m$ subring generated by $Y^{n/(2m)}$. Its value is the same on every useful slot; write this value as $p_F = \sigma_k(p_n) > 0$. Guard-slot errors still affect recovery of the whole polynomial. The $n = 2^{13}$ experiments use a combined lift, trace, and readout routine, reported as Fused in Table 5.

Split: move the ciphertext to the smaller ring. First switch from s_N to the embedded secret $\iota(s_n)$. A ciphertext under this secret can then be split by applying π_j to each component: $\mathsf{Split}_{d,j}(c_0, c_1) = (\pi_j(c_0), \pi_j(c_1))$. Remove the unit before key switching and reapply it after splitting:

$$\widehat{\mathsf{ct}}_j = p_n \mathsf{Split}_{d,j}\left(\mathsf{KS}_{s_N \to \iota(s_n)}(p_N^{-1}\widehat{\mathsf{ct}})\right).$$

The phase is $\pi_j(M) + p_n \pi_j(\kappa_{\mathrm{in}})$, where κ_{in} is the key-switching error. Thus the new error is multiplied by p_n, while the modulus and scale stay unchanged.

Let $J \subseteq \{0, \ldots, d-1\}$ contain the branches to refresh. Following [LD26, Lemma 5], we omit branches whose ideal messages are always zero:

$$j \notin J \quad \Longrightarrow \quad \pi_j(f) = 0 \quad \text{for every } f \in \mathcal{M}.$$

Replace omitted outputs by public zero ciphertexts. The condition concerns the ideal message in each coefficient branch; the ciphertext branch may still contain noise. Padding canonical slots with zeros alone does not ensure it.

StC: move the messages from slots to coefficients. Use Dumezy's slot-to-coefficient transform for the replicated layout [Dum26, Section 5.1]. Let $D_{\mathrm{coef}} = n/(2n_{\mathrm{msg}})$ be the coefficient spacing; it differs from the ring ratio $d = N/n$. After the Trace step below, the ideal message polynomial at unit scale is

$$\sum_{t=0}^{n_{\mathrm{msg}}-1} (\operatorname{Re} z_t + Y^{n/2} \operatorname{Im} z_t) Y^{D_{\mathrm{coef}} t}.$$

The real and imaginary parts therefore occupy coefficients $D_{\mathrm{coef}} t$ and $D_{\mathrm{coef}}(t + n_{\mathrm{msg}})$, respectively. Level ℓ means modulus $\prod_{i=0}^{\ell} q_i$. Our diagonal evaluation and one rescaling step take level 1 to level 0, whose modulus is q_0. Write Δ_{StC} for the scale after this step. Encoding, key-switching, and rescaling errors from this ordinary transform are not automatically multiplied by p_n; we track them separately.

ModRaise: increase the ciphertext modulus. Use sparse-secret encapsulation [BTPH22]: switch to a temporary sparse secret at modulus q_0, lift the centered ciphertext components to the larger modulus Q_{bts}, and switch back to s_n. If $\mu \in R^{(n)}$ represents the phase modulo q_0 before the lift, the lifted phase is $\mu + q_0 I$ for an integer polynomial $I \in R^{(n)}$. The subsequent modular reduction removes this extra multiple of q_0. Switching back to s_n adds another key-switching error.

Trace: keep the message coefficients. The normalized trace of [Dum26] keeps only coefficients whose indices are multiples of D_{coef}:

$$\mathcal{T}_{n_{\mathrm{msg}}}\left(\sum_{i=0}^{n-1} a_i Y^i\right) = \sum_{t=0}^{2n_{\mathrm{msg}}-1} a_{D_{\mathrm{coef}} t} Y^{D_{\mathrm{coef}} t}, \qquad \mathcal{T}_{n_{\mathrm{msg}}}(I) \in R^{(n)}.$$

The remaining carry coefficients are integers, as required for modular reduction. Compute $D_{\mathrm{coef}}\mathcal{T}_{n_{\mathrm{msg}}}$ by rotations and additions, then include the factor $1/D_{\mathrm{coef}}$ in the next transform.

CtS: return the coefficients to slots. The coefficient-to-slot transform reads coefficients $D_{\mathrm{coef}}t$ and $D_{\mathrm{coef}}(t+n_{\mathrm{msg}})$ as real and imaginary parts and restores the replicated layout. Normalize its diagonals by $\Delta_{\mathrm{EM}}/(2q_0K_{\mathrm{EM}}D_{\mathrm{coef}})$, where Δ_{EM} is the scale entering EvalMod and $K_{\mathrm{EM}}>0$ sets the approximation range used there.

We modify diagonal encoding so that its rounding error is multiplied by p_n. Let d_o be the ideal diagonal polynomial at rotation offset o, with zero entries outside the useful slots. Write τ_h for the rotation satisfying $\sigma_k(\tau_h(a))=\sigma_{k+h}(a)$, with indices modulo $n/2$. Here h is the outer rotation assigned to the diagonal in the baby-step/giant-step evaluation. At plaintext encoding scale $\Delta_{\mathrm{diag}}>0$, encode the diagonal as

$$\widetilde{d}_{o,h}=\tau_{-h}(p_n)\,\lfloor\Delta_{\mathrm{diag}}\tau_{-h}(d_o/p_n)\rceil\,,\qquad \frac{\tau_h(\widetilde{d}_{o,h})}{\Delta_{\mathrm{diag}}}=d_o+\frac{p_n\tau_h(\rho_{o,h})}{\Delta_{\mathrm{diag}}},$$

where rounding acts on each coefficient and $\rho_{o,h}=\lfloor\Delta_{\mathrm{diag}}\tau_{-h}(d_o/p_n)\rceil-\Delta_{\mathrm{diag}}\tau_{-h}(d_o/p_n)$. The second identity shows how p_n multiplies the encoding error. Divide useful entries by p_F and apply the rotated unit in both RNS bases Q and P, where P is the auxiliary key-switching modulus. This change needs no extra levels or keys. Errors from rotation keys and modulus descent must still be tracked separately.

ReIm: separate the real and imaginary parts. Form $\mathsf{ct}+\overline{\mathsf{ct}}$ and $-i(\mathsf{ct}-\overline{\mathsf{ct}})$, which double the real and imaginary parts, respectively. The bar denotes homomorphic slot conjugation, and $i=\sqrt{-1}$ is represented by $Y^{n/2}$. The factor $1/2$ in CtS compensates for this doubling. Rescale each channel as $p_n\mathsf{RS}_{D_v}(p_n^{-1}\mathsf{ct})$. Here D_v is the product of removed primes, and RS_{D_v} divides the modulus and scale by D_v with coefficient rounding. The final multiplication by p_n also applies to the rounding error.

EvalMod: remove the integer carries. Set $\beta_{\mathrm{EM}}=q_0/\Delta_{\mathrm{StC}}$. For either channel, let x be the corresponding real or imaginary part of z_t, and let $\nu\in\mathbb{Z}$ be the corresponding coefficient of $\mathcal{T}_{n_{\mathrm{msg}}}(I)$. Ignoring accumulated errors, the normalized input is $(\nu+x/\beta_{\mathrm{EM}})/K_{\mathrm{EM}}$. Choose K_{EM} so that these inputs lie in the polynomial's approximation domain. Use the sine approximation and double-angle circuit of [Dum26] to remove the integer part and recover $x\approx\beta_{\mathrm{EM}}\sin(2\pi x/\beta_{\mathrm{EM}})/(2\pi)$.

Constants take their required values on useful slots and are zero on guards. We use the arithmetic of Section 7.1. In particular, for the three-component product $\widehat{d}=(\widehat{d}_0,\widehat{d}_1,\widehat{d}_2)$, relinearization and rescaling take the forms

$$\widehat{\mathsf{Relin}}(\widehat{d})=p_n\mathsf{Relin}(p_n^{-1}\widehat{d}),\qquad \widehat{\mathsf{RS}}_{D_v}(\widehat{\mathsf{ct}})=p_n\mathsf{RS}_{D_v}(p_n^{-1}\widehat{\mathsf{ct}}).$$

Relinearization converts three components to two under the same secret. These operations multiply their new errors by p_n. Choose output scales using the actual rescaling primes, and recombine the refreshed channels as $\mathsf{ct}_{\mathrm{Re}}+i\mathsf{ct}_{\mathrm{Im}}$. A smaller p_F reduces these errors on useful slots but increases some guard-slot weights. We therefore choose the class count, precision transfer, and scales together [Dum26].

Merge: return to the original ring. For branch ciphertexts $(c_{j,0},c_{j,1})$ under s_n, merge each component using the inverse of the coefficient split: $\mathsf{Merge}_d((c_{j,0},c_{j,1})_j)=(\sum_j X^j\iota(c_{j,0}),\sum_j X^j\iota(c_{j,1}))$. The result is under $\iota(s_n)$. Let $\widehat{\mathsf{ct}}'_j$ be the refreshed branches. Align their moduli and scales, fill omitted branches with public zeros, and switch back to s_N:

$$\widehat{\mathsf{ct}}'_N=p_N\mathsf{KS}_{\iota(s_n)\to s_N}\left(\mathsf{Merge}_d((p_n^{-1}\widehat{\mathsf{ct}}'_j)_j)\right).$$

Its phase is $\sum_{j=0}^{d-1} X^j M'_j(X^d) + p_N \kappa_{\mathrm{ret}}$, where M'_j is the refreshed branch phase, including its error, and κ_{ret} is the return-switch error. The final error bound sums the full embedding error bounds of retained branches, including guards, and the return-switch error $p_N \kappa_{\mathrm{ret}}$. Multiplying by p_N^{-1} gives the main-text representation. We perform this merge immediately after refreshing the retained branches.

D.2 Experiments

We measure time, key storage, output precision, and remaining modulus using Lattigo v6.2.0 [Tun26] and the code of [LD26], running on Intel Xeon Gold 6240R nodes at 2.40 GHz. Tables 4 and 5 report mean times and worst observed useful-slot precision, $-\log_2 \max_j |\widehat{z}_j - z_j|$. Each configuration uses three independent key sets and three inputs per key, except Group III, which uses sixteen inputs per key. Parallel runs process the real and imaginary EvalMod channels concurrently. Precision and output level are measured after return to the original ring; $L_{\mathrm{out}} = \ell$ means that $\ell + 1$ RNS primes remain.

Groups I, II, and III have original ring degrees $N = 2^{17}$, 2^{16}, and 2^{13}, respectively; n is the degree used during bootstrapping. Every configuration uses eight precision classes, $K_{\mathrm{EM}} = 17$, and a temporary secret of Hamming weight 32. The $n = 2^{13}$ cases use main secrets of Hamming weight 1024; the others use uniform ternary main secrets. The Security row gives the smallest classical estimate over the evaluated primal, dual, and hybrid attacks and all main and temporary secrets. We use Lattice Estimator [APS15], commit `601905601`, with `RC.MATZOV`, $\sigma_{\mathrm{err}} = 3.2$, unbounded samples, and $q = 2^{\lceil \log_2(QP) \rceil}$. Fixed-weight secrets use its balanced ternary model.

Table 4: Main CKKS parameters.

Metric	Group I		Group II			Group III
	$L_{\mathrm{out}} = 13$	$L_{\mathrm{out}} = 1$	$L_{\mathrm{out}} = 13$	$L_{\mathrm{out}} = 13$	$L_{\mathrm{out}} = 1$	$L_{\mathrm{out}} = 2$
$\log N$	17	17	16	16	16	13
$\log n$	15	13	16	15	14	13
Useful branches	1	1	1	1	1	1
Messages n_{msg}	8	1	8	8	8	1
L_{in}	1	0	1	1	1	0
$\log \Delta$ (in/EM)	40/26	22/20	40/25	40/25	40/25	20/20
$\log \Delta_{\mathrm{out}}$	30.000	22.000	30.000	30.000	30.000	17.000
$\log Q_{\mathrm{in}}$	80.00	23.75	80.00	80.00	80.00	19.98
$\log Q_{\mathrm{bts}}$	736.83	194.94	728.32	728.32	368.11	188.90
$\log Q_{\mathrm{out}}$	440.30	46.21	440.21	440.21	80.00	53.95
$\log(PQ_{\mathrm{bts}})$	846.83	210.94	838.32	838.32	394.11	204.90
Security. (bits)	131.19	129.35	154.49	131.19	131.42	133.16

The first two columns of Group II isolate the effect of switching from degree 2^{16} to 2^{15}. Both use precision allocation with the same messages, primes, public unit, and output level. The other columns test different parameter choices. The $n \geq 2^{14}$ cases use the replicated layout with $\Re z, \Im z \in [-0.1, 0.1]$. The direct and ring-switched $n = 2^{13}$ cases use $|z| = 1$ and $|z| \leq 1/2$, respectively, so they are not comparisons on identical inputs.

For the matched Group II configurations, switching from degree 2^{16} to 2^{15} reduces parallel time from 25.403 to 12.065 s and server key storage from 7.011 to 3.466 GiB. Both return approximately 27-bit precision at output level 13. For these inputs and parameters, the savings therefore include

Table 5: CKKS bootstrapping performance for the configurations in Table 4. Each ring-switching configuration refreshes only one publicly useful branch and immediately returns to the original ring at L_{out}.

	Group I		**Group II**			**Group III**
	$L_{\text{out}} = 13$	$L_{\text{out}} = 1$	$L_{\text{out}} = 13$	$L_{\text{out}} = 13$	$L_{\text{out}} = 1$	$L_{\text{out}} = 2$
	Time (sec)					
Split	0.114	0.094	0.001	0.049	0.072	0.013
StC	0.043	–	0.095	0.038	0.052	–
ModRaise†	4.203	–	9.416	4.005	1.537	–
CtS	1.850	–	3.767	1.766	0.723	–
Fused†	–	1.967	–	–	–	1.942
EvalMod	5.825	0.498	12.116	5.735	1.308	0.532
Merge	1.021	0.171	0.008	0.472	0.072	–
Serial‡	18.011	3.279	36.038	16.998	4.994	2.891
Parallel‡	13.056	2.733	25.403	12.065	3.764	2.488
	Precision (bits)					
	27.09	6.11	27.07	27.10	27.05	3.13
	Key Generation (sec)*					
	31.37	5.43	55.52	28.27	16.74	4.90
	Server Key Size (GiB)**					
	3.580	0.794	7.011	3.466	2.043	0.696

† ModRaise includes Trace and SSE; CtS includes ReIm. Fused denotes combined lift/trace/readout.
‡ Serial/parallel times cover the complete bootstrap using one/two workers; stage times use parallel mode. Split and Merge include representation conversions at entry and return, respectively, and their key switches when present. Setup, encryption, decryption, verification, and serialization are excluded.
∗ Key-generation time includes evaluator setup and excludes bootstrap matrix encoding.
∗∗ Server key size counts serialized bootstrapping evaluation material.

splitting and merging, without reducing precision or output levels. Only the branch containing useful messages is refreshed. Starting from degree 2^{17} and bootstrapping at degree 2^{15} also returns approximately 27-bit precision at level 13, in 13.056 s with parallel evaluation.

Using degree 2^{14} still gives approximately 27-bit precision, but only at output level 1. The smaller modulus budget therefore supports accurate outputs with fewer levels left for further computation. High output precision therefore does not by itself mean that many levels remain for further computation.

At degree 2^{13}, the tested circuits refresh a single message with only a few bits of output precision. Direct bootstrapping takes 2.488 s in parallel mode and returns 3.13-bit precision at level 2, with $Q_{\text{out}} > Q_{\text{in}}$. Switching from degree 2^{17} to 2^{13} returns 6.11-bit precision at level 1 in 2.733 s. These examples show that bootstrapping can increase the modulus available for further computation even at this small degree, with lower output precision than in the larger-ring experiments.

E A Concrete Example of the Error Bounds

We compute the bounds for the precision requirements $(160, 32, 32, 32)$, apply the recovery condition, and compare the bounds with a recorded execution.

Parameters. With $N = 2^{15}$ and $r = 2^{14}$, group $g \in \{0, 1, 2, 3\}$ contains the 2^{12} slots $j \equiv g \pmod 4$. Inputs lie in $[3/4, 7/8]$; four successive squarings compute x_j^{16}, with the public correction by u before each relinearization. Put $Y = X^{4096}$ and use

$$\begin{aligned} u = & -3645862971 + 834730735(Y - Y^7) \\ & - 173731438(Y^2 - Y^6) + 3069345111(Y^3 - Y^5). \end{aligned}$$

For $j \equiv g \pmod 4$, write $\omega_g = |\sigma_j(u)|$. Certified enclosures give the displayed logarithms

$$(\log_2 \omega_g)_{g=0}^3 \approx (-96.435365,\ 30.604317,\ 32.857700,\ 32.973348).$$

Let $Q_0 = \prod_{i=0}^9 q_i$, with $q_i = 2^{\ell_{q,i}} - c_{q,i}$ and

$$\begin{aligned} (\ell_{q,0}, \ldots, \ell_{q,9}) &= (49, 48, 47, 46, 47, 46, 47, 46, 47, 46), \\ (c_{q,0}, \ldots, c_{q,4}) &= (720895, 393215, 1048575, 589823, 1638399), \\ (c_{q,5}, \ldots, c_{q,9}) &= (1769471, 1835007, 3407871, 2490367, 3735551). \end{aligned}$$

The auxiliary primes are $(p_i)_{i=0}^3 = 2^{28} - (65535, 196607, 393215, 1376255)$, componentwise. Set $P = \prod_i p_i$ and $p_\star = p_0$. Encryption uses $p_\star$; relinearization uses P, grouping active q_i into consecutive blocks of at most four primes. At layer $t = 1, \ldots, 4$, divide successively by $d_{t,1} = q_{11-2t}$ and $d_{t,2} = q_{10-2t}$, giving $D_t = d_{t,1} d_{t,2}$ and $Q_t = \prod_{i=0}^{9-2t} q_i$.

The Lattigo v6.2.0 execution [Tun26] uses $\Delta_0 = 2^{93}$ and 1024-bit encoding, scale arithmetic, and decoding. The stored Δ_t is obtained by squaring Δ_{t-1} and dividing successively by $d_{t,1}, d_{t,2}$, rounding after each operation. Define $\chi_t = \Delta_{t-1}^2/(D_t \Delta_t)$; here $|\chi_t - 1| < 2^{-1023}$. Rational interval propagation bounds the inverse-transform coefficient error by 2^{-788}, the subsequent floating-point error in scaling and rounding by 2^{-818}, and the final decoder error by 2^{-734} per slot, including weight approximation.

Auxiliary bounds. For this ring, use $\|f\|_{\sigma,\infty} \le \overline{\Gamma}_N \|f\|_{\mathrm{coef},\infty}$ with $\overline{\Gamma}_N = 20861 \ge \Gamma_N$, as permitted by (4). Secret and encryption randomizer coefficients are uniform ternary. Gaussian error coefficients have sampler parameter 3.2, truncation parameter 19.2, and conservative absolute bound 20. We condition on $\|s\|_{\sigma,\infty} \le B_s = 3473$ and $\|e_k\|_{\sigma,\infty} \le B_e = 70202$ for evaluation-key errors. Hoeffding and union bounds give these constants as

$$B_s = \left\lceil \sqrt{2N(160 + 24)} \right\rceil, \qquad B_e = \left\lceil 20\sqrt{2N(160 + 28)} \right\rceil,$$

covering 256 ternary and 4096 error polynomials with joint failure probability at most 2^{-159}. The archived security estimate for $Q_0 P$ exceeds the separate 128-bit security target. For binary64 CRT conversion from n source primes, set

$$\delta_{\mathrm{CRT}}(n) = \frac{n(n+3)2^{-53}}{1 - (n+3)2^{-53}}.$$

The bounds below assume round-to-nearest conversion and exact integer modular operations within their checked ranges. They cover all actual operation inputs, including those after public correction.

Explicit Bounds for the Example Encoding error B_{enc}. Including coefficient rounding and the numerical errors above,

$$\|e_{\mathrm{enc}}\|_{\mathrm{coef},\infty} \le \tfrac{1}{2} + \Delta_0\, 2^{-788} + 2^{-818} < 1, \qquad B_{\mathrm{enc}} = \overline{\Gamma}_N.$$

Encryption error B_{dec}. The sampled errors after division by $p_\star$ and the two modulus-down errors give, respectively,

$$B_{\mathrm{dec}} = \frac{20\overline{\Gamma}_N(\overline{\Gamma}_N + 1 + B_s)}{p_\star} + \overline{\Gamma}_N(1 + B_s)\left(\frac{1}{2} + \delta_{\mathrm{CRT}}(1) + \frac{1}{2p_\star}\right).$$

Relinearization error K_t. Let $Q_{t,k}$ and $m_{t,k}$ be the prime product and prime count of block k in Q_{t-1}. The implementation's digit coefficient bound is

$$H_{t,k} = (Q_{t,k} + 1)/2 \quad (m_{t,k} = 1), \qquad H_{t,k} = Q_{t,k}(1/2 + \delta_{\mathrm{CRT}}(m_{t,k})) + 1/2 \quad (m_{t,k} > 1).$$

Including centering and CRT errors, (22) gives

$$K_t = \frac{\overline{\Gamma}_N B_e}{P} \sum_k H_{t,k} + \overline{\Gamma}_N(1 + B_s)\left(\frac{1}{2} + \delta_{\mathrm{CRT}}(4) + \frac{1}{2P}\right).$$

Rescaling residues R_t. Each prime division has phase error at most $\overline{\Gamma}_N(1 + B_s)/2$. The second division also divides the first error by $d_{t,2}$, so the aggregate numerator bound is

$$R_t = D_t\, \frac{\overline{\Gamma}_N(1 + B_s)}{2}\left(1 + \frac{1}{d_{t,2}}\right).$$

Message amplitude $A_{t,j}$. Since $x_{t,j} = x_{0,j}^{2^t}$, take $A_{t,j} = (7/8)^{2^t}$ for $0 \le t \le 4$.

Initial error bound $B_{0,j}$. The fresh encoding and encryption errors give

$$B_{0,j} = \frac{B_{\mathrm{enc}} + B_{\mathrm{dec}}}{\Delta_0}.$$

Accumulated error bound $B_{t,j}$. Lemma 7.2, with the stored-scale correction, gives for $j \equiv g \pmod 4$

$$B_{t,j} = |\chi_t|(2A_{t-1,j}B_{t-1,j} + \omega_g B_{t-1,j}^2) + \frac{K_t + R_t}{D_t\Delta_t} + \frac{|\chi_t - 1|A_{t-1,j}^2}{\omega_g}.$$

The last term accounts for scale rounding; exact scales give $\chi_t = 1$. Computations use weight enclosures and outward rounding. The implementation propagates $\omega_g B_{t,j}$; division by a certified lower bound on ω_g recovers a valid error bound.

Output error bound $B_{\mathrm{out},j}$. Restoration is exact, so $B_{\mathrm{out},j} = B_{4,j}$. Numerical decoding adds at most 2^{-734} to the restored error bound.

Applying the Recovery Condition Recovery at the selected modulus. All ideal outputs are nonnegative real numbers, so the constant coefficient attains the inverse-embedding coefficient norm:

$$\|\sigma^{-1}(\mathbf{x}_{\mathrm{out}})\|_{\mathrm{coef},\infty} = r^{-1}\sum_j x_{0,j}^{16} \le (7/8)^{16} \approx 2^{-3.082321}.$$

Write $\overline{\omega}(u) := r^{-1}\sum_j |\sigma_j(u)|$ for the average embedding weight. The propagated coefficient error term is approximately $2^{-33.275980}$; $B_{\mathrm{out}}\overline{\omega}(u)$ has the same displayed logarithm. Groups $1, 2, 3$ contribute 9.15%, 43.61%, and 47.25% of the error term, showing that low-precision outputs still enter recovery. For $Q_4 = q_0 q_1$, the normalized coefficient limit for recovery is

$$(Q_4 - 1)/(2\Delta_4) \approx 2^{2.999999}.$$

The message-plus-error bound uses only $2^{-6.082320}$ of this range. Thus (7) guarantees recovery for all allowed inputs on the stated event; the ideal message term dominates.

Effect on modulus selection. Dropping q_1 after evaluation leaves the formal precision bounds unchanged, but reduces the available range to $2^{-45.000001}$. The ideal message term for the recorded inputs is $2^{-4.463161}$, so this candidate fails the recovery check. At the fixed scale, the sufficient condition requires $\log_2 Q_{\mathrm{out}} > 89.537$ approximately; the selected $\log_2 Q_4 \approx 97$ passes. This is a sufficient-condition check, not a global optimality claim. The selected parameters also certify 161.766, 34.727, 32.473, and 32.358 bits in the four groups, meeting every precision target.

Bounds and Observed Values The secret key from the recorded run is used to extract phase errors and rescaling residues; restored errors are measured against exact rational reference values. The absolute values of the embeddings are recorded at 1024-bit precision, while the recovered coefficient maximum is measured exactly. Weight approximation is included in the bounds and does not affect the displayed digits. In Table 6, o is the observed maximum, B its bound, and the gap is $\log_2(B/o)$. Write $\mathcal{I}_g = \{j \in [r-1] : j \equiv g \pmod 4\}$ and $B_{\mathrm{out},g}$ for the common bound in group g. Checks use unrounded values.

Table 6: Observed quantities and explicit bounds for the four-squaring example. Unqualified maxima are over all r slots. The last row uses the coefficient bound valid for all allowed inputs, not the sharper bound specific to the recorded inputs.

Quantity	**Bound**	**Observed** $\log_2 o$	**Upper bound** $\log_2 B$	**Gap (bits)** $\log_2(B/o)$
$\|e_{\mathrm{enc}}\|_{\sigma,\infty}$	B_{enc}	7.696	14.349	6.653
$\|e_{\mathrm{dec}}\|_{\sigma,\infty}$	B_{dec}	15.539	25.111	9.572
$\|\kappa_1\|_{\sigma,\infty}$	K_1	95.539	107.546	12.007
$\|\kappa_2\|_{\sigma,\infty}$	K_2	95.491	107.546	12.055
$\|\kappa_3\|_{\sigma,\infty}$	K_3	95.371	107.459	12.088
$\|\kappa_4\|_{\sigma,\infty}$	K_4	95.268	107.459	12.191
$\|\rho_{1,0} + \rho_{1,1}s\|_{\sigma,\infty}$	R_1	108.540	118.111	9.571
$\|\rho_{2,0} + \rho_{2,1}s\|_{\sigma,\infty}$	R_2	108.429	118.111	9.682
$\|\rho_{3,0} + \rho_{3,1}s\|_{\sigma,\infty}$	R_3	108.260	118.111	9.851
$\|\rho_{4,0} + \rho_{4,1}s\|_{\sigma,\infty}$	R_4	108.702	118.111	9.409
$\max_j \|x_{4,j}\|$	$A_{4,j}$	-3.082	-3.082	0.000
$\max_j \|\eta_{0,j}\|$	$\max_j B_{0,j}$	-77.461	-67.888	9.572
$\max_j \|\eta_{1,j}\|$	$\max_j B_{1,j}$	-76.706	-66.429	10.277
$\max_j \|\eta_{2,j}\|$	$\max_j B_{2,j}$	-75.713	-65.507	10.206
$\max_j \|\eta_{3,j}\|$	$\max_j B_{3,j}$	-75.600	-65.059	10.541
$\max_{j\in\mathcal{I}_0} \|\eta_{4,j}\|$	$B_{\mathrm{out},0}$	-75.980	-65.331	10.649
$\max_{j\in\mathcal{I}_1} \|\eta_{4,j}\|$	$B_{\mathrm{out},1}$	-76.965	-65.331	11.634
$\max_{j\in\mathcal{I}_2} \|\eta_{4,j}\|$	$B_{\mathrm{out},2}$	-76.578	-65.331	11.247
$\max_{j\in\mathcal{I}_3} \|\eta_{4,j}\|$	$B_{\mathrm{out},3}$	-76.931	-65.331	11.600
$\max_j \|\eta_{4,j}\|$	B_{out}	-75.980	-65.331	10.649
$\max_{j\in\mathcal{I}_0} \|e_{4,j}\|$	$\omega_0 B_{\mathrm{out},0}$	-172.415	-161.766	10.649
$\max_{j\in\mathcal{I}_1} \|e_{4,j}\|$	$\omega_1 B_{\mathrm{out},1}$	-46.361	-34.727	11.634
$\max_{j\in\mathcal{I}_2} \|e_{4,j}\|$	$\omega_2 B_{\mathrm{out},2}$	-43.721	-32.473	11.247
$\max_{j\in\mathcal{I}_3} \|e_{4,j}\|$	$\omega_3 B_{\mathrm{out},3}$	-43.958	-32.358	11.600
$\|M_4\|_{\mathrm{coef},\infty}/\Delta_4$	*All allowed inputs*	-4.463	-3.082	1.381

F Concurrent Work

Updated on October 22, 2026. An early draft of this work was posted on GitHub on October 11, 2026 (commit record). We present these concurrent works below in alphabetical order by their first authors' surnames.

We currently provide an overview of the three concurrent works and will add a detailed comparison of our work with these three papers in a future revision.

Cheon and Lee [CL26] use cyclotomic units to trade unused capacity for precision and, like us, support general precision requirements across slots. Their existence theorem [Theorem 1] bounds approximation error by the covering radius; our independent Fourier construction approximates the same targets with its own explicit absolute error bound (our Theorem 4.1). They construct units using mixed-integer linear programming and develop corresponding rescaling and key-switching procedures. Their bootstrapping procedures reduce precision-dependent modulus consumption, including the base modulus in one variant.

Dumezy [Dum26] targets small batches, trading unused slots for higher precision or a smaller scale and lower modulus consumption at fixed precision. He develops rounding procedures for encoding, rescaling, and key switching, and constructs units using mixed-integer linear programming. He establishes circuit correctness and error bounds for unused slots, and quantifies limits on precision gains [Theorems 2–3]. He also provides bootstrapping, conversions to and from CKKS, and an implementation. He also provides an IND-CPA security reduction and discusses noise flooding for IND-CPA^{D} security.

Ma et al. [MLWW26] build on cyclotomic-integer rounding and construct public ring elements, not necessarily units, from short lattice vectors [Lemma 4]. Their construction trades useful slots for lower modulus consumption and higher precision. Like us, they largely retain standard CKKS operations and analyze error control on unused slots [Lemmas 5–6]. They also compare unit and nonunit elements obtained by their search methods. They validate the error bounds experimentally and demonstrate high-precision arithmetic in a prototype implementation. Retaining sufficient precision after noise flooding, including when it is used for IND-CPA^{D} security, also motivates their high-precision setting.

References

[ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum Key Exchange—A New Hope. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 327–343, Austin, TX, August 2016. USENIX Association.

[APS15] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

[BBB+17] Charlotte Bonte, Carl Bootland, Joppe W. Bos, Wouter Castryck, Ilia Iliashenko, and Frederik Vercauteren. Faster homomorphic function evaluation using non-integral base encoding. In *Cryptographic Hardware and Embedded Systems – CHES 2017*, volume 10529 of *Lecture Notes in Computer Science*, pages 579–600. Springer, 2017.

[BCC+22] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2022.

[BCC+25] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and Donggeon Yhee. Security guidelines for implementing homomorphic encryption. *IACR Communications in Cryptology*, 1(4), 2025.

[BCIV20] Carl Bootland, Wouter Castryck, Ilia Iliashenko, and Frederik Vercauteren. Efficiently processing complex-valued data in homomorphic encryption. *Journal of Mathematical Cryptology*, 14(1):55–65, 2020.

[BGGJ20] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. CHIMERA: Combining ring-LWE-based fully homomorphic encryption schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.

[BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *Innovations in Theoretical Computer Science (ITCS)*, pages 309–325. ACM, 2012.

[BK25] Dan Boneh and Jaehyung Kim. Homomorphic encryption for large integers from nested residue number systems. In *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 338–370. Springer, 2025.

[Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer, 2012.

[BTPH22] Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In *Applied Cryptography and Network Security – ACNS 2022*, volume 13269 of *Lecture Notes in Computer Science*, pages 521–541. Springer, 2022.

[BV11] Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from Ring-LWE and security for key dependent messages. In *Advances in Cryptology – CRYPTO 2011*, pages 505–524. Springer, 2011.

[CCK+25] Jung Hee Cheon, Hyeongmin Choe, Minsik Kang, Jaehyung Kim, Seonghak Kim, Johannes Mono, and Taeyeong Noh. Grafting: Decoupled scale factors and modulus in RNS-CKKS. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1098–1112. ACM, 2025.

[CCKS23] Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 696–710. ACM, 2023.

[CCS19] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 34–54. Springer, 2019.

[CDPR16] Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering short generators of principal ideals in cyclotomic rings. In *Advances in Cryptology – EUROCRYPT 2016*, pages 559–585. Springer, 2016.

[CHK+18] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2018*, volume 10820 of *Lecture Notes in Computer Science*, pages 360–384. Springer, 2018.

[CHK+19] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full RNS variant of approximate homomorphic encryption. In *Selected Areas in Cryptography – SAC 2018*, volume 11349 of *Lecture Notes in Computer Science*, pages 347–368. Springer, 2019.

[CIL21] Hao Chen, Ilia Iliashenko, and Kim Laine. When HEAAN meets FV: a new somewhat homomorphic encryption with reduced memory overhead. In *Cryptography and Coding (IMACC 2021)*, volume 13129 of *Lecture Notes in Computer Science*, pages 265–285. Springer, 2021.

[CIV18] Wouter Castryck, Ilia Iliashenko, and Frederik Vercauteren. Homomorphic $\mathrm{SIM}^2\mathrm{D}$ operations: Single instruction much more data. In *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 338–359. Springer, 2018.

[CK25] Jean-Sébastien Coron and Robin Köstler. Low-latency bootstrapping for CKKS using roots of unity. Cryptology ePrint Archive, Paper 2025/651, 2025.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437. Springer, 2017.

[CKSS25] Hyeongmin Choe, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Leveraging discrete CKKS to bootstrap in high precision. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2025.

[CL26] Jung Hee Cheon and Junho Lee. High-precision CKKS with focal units. Cryptology ePrint Archive, Paper 2026/2531, 2026.

[CLPX18] Hao Chen, Kim Laine, Rachel Player, and Yuhou Xia. High-precision arithmetic in homomorphic encryption. In *Topics in Cryptology – CT-RSA 2018*, pages 116–136. Springer, 2018.

[CPL26] Gyeongwon Cha, Dongjin Park, and Joon-Woo Lee. Improved radix-based approximate homomorphic encryption for large integers via lightweight bootstrapped digit carry. In *Advances in Cryptology – EUROCRYPT 2026*. Springer, 2026.

[CS25] Jean-Sébastien Coron and Tim Seuré. PaCo: Bootstrapping for CKKS via partial CoeffToSlot. In *Advances in Cryptology – ASIACRYPT 2025, Part VII*, volume 16251 of *Lecture Notes in Computer Science*, pages 36–67. Springer, 2025.

[CSV17] Anamaria Costache, Nigel P. Smart, and Srinivas Vivek. Faster homomorphic evaluation of discrete Fourier transforms. In *Financial Cryptography and Data Security*. Springer, 2017.

[DKRS03] Irit Dinur, Guy Kindler, Ran Raz, and Shmuel Safra. Approximating CVP to within almost-polynomial factors is NP-hard. *Combinatorica*, 23(2):205–243, 2003.

[Dum26] Jules Dumezy. Fully homomorphic encryption for small batches of complex numbers. Cryptology ePrint Archive, Paper 2026/2513, 2026.

[EHKS14] Kirsten Eisenträger, Sean Hallgren, Alexei Kitaev, and Fang Song. A quantum algorithm for computing the unit group of an arbitrary degree number field. In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing (STOC)*, pages 293–302. ACM, 2014.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178. ACM, 2009.

[GHPS12] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P. Smart. Ring switching in BGV-style homomorphic encryption. In Ivan Visconti and Roberto De Prisco, editors, *Security and Cryptography for Networks – SCN 2012*, volume 7485 of *Lecture Notes in Computer Science*, pages 19–37, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[GV25] Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption for cyclotomic prime moduli. In *Advances in Cryptology – EUROCRYPT 2025*, volume 15603 of *Lecture Notes in Computer Science*, pages 366–397. Springer, 2025.

[HHC19] Kyoohyung Han, Minki Hhan, and Jung Hee Cheon. Improved homomorphic discrete Fourier transforms and FHE bootstrapping. *IEEE Access*, 7:57361–57370, 2019.

[HK20] Kyoohyung Han and Dohyeong Ki. Better bootstrapping for approximate homomorphic encryption. In *Topics in Cryptology – CT-RSA 2020*, volume 12006 of *Lecture Notes in Computer Science*, pages 364–390. Springer, 2020.

[HS14] Shai Halevi and Victor Shoup. Algorithms in HElib. In *Advances in Cryptology – CRYPTO 2014, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 554–571. Springer, 2014.

[HS18] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in HElib. Cryptology ePrint Archive, Paper 2018/244, 2018. Presented at CRYPTO 2018.

[KCY25] Jihwan Kim, Jung Hee Cheon, and Yongdong Yeo. OverModRaise: Reducing modulus consumption of CKKS bootstrapping. Cryptology ePrint Archive, Paper 2025/1298, 2025. Published in IACR Communications in Cryptology 2025.

[Kim25] Jaehyung Kim. Efficient homomorphic integer computer from CKKS. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):873–898, 2025.

[KPK+22] Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. EvalRound algorithm in CKKS bootstrapping. In *Advances in Cryptology – ASIACRYPT 2022*, pages 161–187. Springer, 2022.

[LCK+23] Yongwoo Lee, Seonyoung Cheon, Dongkwan Kim, Dongyoon Lee, and Hanjun Kim. ELASM: Error-latency-aware scale management for fully homomorphic encryption. In *32nd USENIX Security Symposium*, pages 4697–4714. USENIX Association, 2023.

[LD26] Zhaoyang Liang and Dan Ding. Bootstrapping via ring switching without slot recovery. Cryptology ePrint Archive, Paper 2026/1779, 2026.

[LHC+22] Yongwoo Lee, Seonyeong Heo, Seonyoung Cheon, Shinnung Jeong, Changsu Kim, Eunkyung Kim, Dongyoon Lee, and Hanjun Kim. HECATE: Performance-aware scale optimization for homomorphic encryption compiler. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–204, 2022.

[LLK+22] Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 551–580. Springer, 2022.

[LLL+21] Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 618–647. Springer, 2021.

[LN13] Mingjie Liu and Phong Q. Nguyen. Solving BDD by enumeration: An update. In Ed Dawson, editor, *Topics in Cryptology – CT-RSA 2013*, pages 293–309, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.

[MAT22] MATZOV. Report on the security of LWE: Improved dual lattice attack. Technical report, Israel Defense Forces, Israel, April 2022.

[MHWW26] Shihe Ma, Tairong Huang, Anyu Wang, and Xiaoyun Wang. Practical dense-key bootstrapping with subring secret encapsulation. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025, Part VII*, volume 16251 of

Lecture Notes in Computer Science, pages 101–130, Singapore, 2026. Springer Nature Singapore.

[MLWW26] Shihe Ma, Jiaming Liu, Anyu Wang, and Xiaoyun Wang. Homomorphic encryption for high-precision arithmetic of approximate numbers. Cryptology ePrint Archive, Paper 2026/2524, 2026.

[PRSD17] Chris Peikert, Oded Regev, and Noah Stephens-Davidowitz. Pseudorandomness of Ring-LWE for any ring and modulus. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 461–473. ACM, 2017.

[PZZ26] Chris Peikert, Doron Zarchy, and Guy Zyskind. High-precision exact FHE made simple, general, and fast. In *Advances in Cryptology – CRYPTO 2026, Part II*, volume 16801 of *Lecture Notes in Computer Science*, pages 525–557. Springer, 2026.

[SSKM25] Hyewon Sung, Sieun Seo, Taekyung Kim, and Chohong Min. EvalRound+ bootstrapping and its rigorous analysis for CKKS scheme. *IEEE Access*, 13:140847–140866, 2025.

[SV14] Nigel P. Smart and Frederik Vercauteren. Fully homomorphic SIMD operations. *Designs, Codes and Cryptography*, 71(1):57–81, 2014.

[Tun26] Tune Insight SA. Lattigo: Lattice-based multiparty homomorphic encryption in Go. Software, version 6.2.0, 2026.

[Was97] Lawrence C. Washington. *Introduction to Cyclotomic Fields*, volume 83 of *Graduate Texts in Mathematics*. Springer, New York, 2 edition, 1997.

[YZC+26] Lianglin Yan, Pengfei Zeng, Heyang Cao, Peizhe Song, and Mingsheng Wang. Faster bootstrapping for CKKS with less modulus consumption. In *Public-Key Cryptography – PKC 2026, Part IV*, volume 16554 of *Lecture Notes in Computer Science*, pages 335–366. Springer, 2026.

[Zhe26] Xiaopeng Zheng. Faster CoeffToSlot and SlotToCoeff for sparsely packed ciphertexts with application to CKKS bootstrapping. Cryptology ePrint Archive, Paper 2026/1023, 2026. To appear in ASIACRYPT 2026.